



MODULE: STW7089CEM — Introduction to
Statistical Methods for Data Science



Modeling EEG Signals Using Polynomial Regression

*Applying Statistical Modeling to Understand
Brainwave Patterns*



Student Name : Rohit Jha



Coventry University ID : 11782276



College ID (Softwarica) : 210178



Lecturer : Ujjwal Koirala

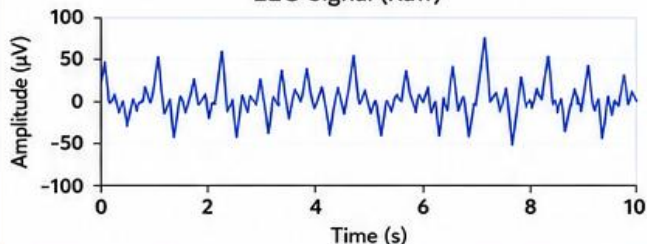


Institution : Softwarica College of IT & E-Commerce
in collaboration with Coventry University

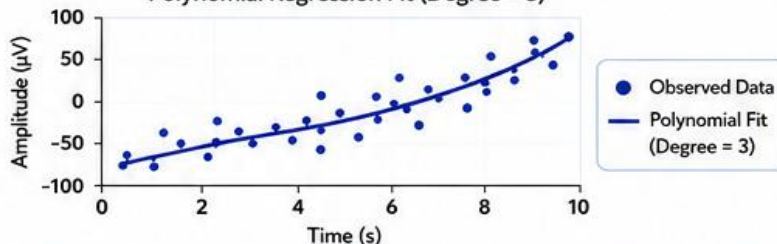


Date : July 2026

EEG Signal (Raw)



Polynomial Regression Fit (Degree = 3)



KEY TOPICS



Data Science



Statistical Analysis



AI & Machine
Learning



R Programming &
Statistical Modeling



EEG Analysis

Table of Contents

Introduction.....	3
Task 1: Preliminary Data Analysis	4
1.1 Time Series Plots	4
1.2 Distribution Analysis	6
1.3 Analysis of Correlation and Scatter Plots	9
1.4 Fitting a Linear Regression Model	11
Task 2: Regression Modelling	12
Candidate Models	12
Task 2.1: Least Squares Parameter Estimations	13
Task 2.2: Residual Sum of Squares (RSS).....	14
Task 2.3: Log-Likelihood	14
Task 2.4: AIC and BIC	15
Task 2.5: Residual Distribution Analysis	16
Task 2.6: Model Selection	18
Task 2.7: Train/Test Split and Prediction Confidence Intervals.....	19
Task 3: Approximate Bayesian Computation (ABC)	21
3.1 Methodology	21
3.2 Parameter Selection	22
3.3 Prior Specification and ABC Execution	22
3.4 Posterior Distributions	23
3.5 Interpretation of ABC Results	24
Discussion.....	25
Conclusion	27
References.....	28
GitHub Link:.....	29
Appendix: R Code.....	29
# TASK 1: PRELIMINARY DATA ANALYSIS.....	29
# TASK 2: REGRESSION MODELLING	32
# TASK 3: APPROXIMATE BAYESIAN COMPUTATION (ABC)	35

Lists of Figures

Figure 1: Overlaid time-series comparison of all input EEG signals (X).....	5
Figure 2: Time-Series of the Four EEG Input Signals.....	5
Figure 3: Time-series of the output EEG signal (Y).....	6
Figure 4: Distribution of all Four X inputs	7
Figure 5: Histograms with Kernel Density Estimates (X1–X4).	7
Figure 6: Distribution of Output Signal (Y).	8
Figure 7: Pearson correlation Matrix of EEG Signals.	9
Figure 8: Bivariate relationship analysis between individual input signals (X1-X4) and the output signal (Y).	9
Figure 9: Pairwise scatter plots of input EEG signals.....	10
Figure 10: Linear model diagnostics. Actual vs. fitted values. & Residuals vs. fitted values.	11
Figure 11: Histograms of Model Residuals with Overlaid Normal Density Curves. ..	17
Figure 12: Q-Q plots of residuals in the five proposed models.	17
Figure 13: Test Set Predictions with 95% Intervals of Model 2 (error bars).....	20
Figure 14: Joint and marginal posterior distributions from rejection ABC.	23

Lists of Table

Table 1: Sample Data.....	3
Table 2: Sample data of X inputs and time.....	4
Table 3: In this section, an initial exploratory analysis of the EEG signals was performed to understand their temporal dynamics, distributional properties, and relationships between signals. The analysis also implemented a baseline linear regression model.	6
Table 4: Summary statistics for each EEG signal.....	8
Table 5: Pearson correlation matrix of all EEG signals.....	10
Table 6: Linear regression model coefficients, standard errors, t-statistics, and p-values.	11
Table 7: Linear regression model performance metrics.	12
Table 8: Least Squares Parameter Estimates for Candidate Models.	13
Table 9: Residual Sum of Squares of Candidate Models.	14
Table 10: Estimated variance and log-likelihood for each model.	15
Table 11: Comparison of Candidate Regression Models.	16
Table 12: Shapiro–Wilk Test Results for the Five Candidate Models.	18
Table 13: Parameter estimates from full dataset versus training data, with absolute differences.....	19
Table 14: Prediction performance metrics on training and testing data for the selected Model 2.	20
Table 15: Parameter Selection for Rejection ABC.....	22
Table 16: Summary of ABC Posterior Estimates.	23

Introduction

Electroencephalography (EEG) is a commonly utilized non-invasive measure of the brain's electrical activity and its macro-scale neurophysical dynamics ([Nunez & Srinivasan, 2006](#)). When considering methods for brain imaging such as functional magnetic resonance imaging (fMRI) there is no question that electroencephalography (EEG) is much cheaper and provides faster insights into brain activity. EEG is affordable and gives a lot of information, which is why doctors and researchers prefer it. That makes EEG suitable for their work and research. ([Sanei & Chambers, 2007](#)). EEG-based analysis and modelling techniques have been widely used to characterize a variety of neurological disorders such as the Parkinson's disease, epilepsy and tremor, as well as the development of brain-computer interfaces (BCI).

A key area of research in the study of neuroscience is the determination of interconnections among different areas of the brain. This can occur through modeling and analyzing the different signals produced by EEG channels. This session's objective is to find the optimal polynomial regression model that can accurately represent the connection among four EEG input signals and one EEG output signal among five non-linear regression models. In total, there are five sets of time series data that represent EEG signals that number 201 samples. The output signal (y) and four of the EEG input signals (x1, x2, x3, and x4) are sampled for a total of 0.4 seconds at a frequency of 500 Hz. All of the signals include noise in the form of additive independent identically distributed (IID) Gaussian noise containing a variable amount of unknown variance and a mean value of zero. In addition to this, the EEG data can also be analyzed using the most appropriate statistical methodology for the filtering of a great deal of extraneous noise and to extract the true electrical neurophysiological signals. Our new and advanced software enables us to discover patterns that are undetectable through conventional means. This analytical process is essential in the study of neurological disorders and is also a factor in increasing the reliability of brain-computer interfaces.

time	X1	X2	X3	X4	y
0.002	0.3491160	1.91906000	1.0049500	0.5778450	0.321224404
0.004	-1.0097000	0.27246100	-0.9122970	-2.6973800	0.138223937
0.006	-0.8076080	-1.58148000	0.1943780	-1.6222700	0.359189812
0.008	2.0792500	0.74281200	1.9280900	1.5582200	0.751551701
0.010	3.2492000	1.94461000	1.7990000	2.3810600	1.417170927
0.012	4.2155200	0.76991200	-0.5173890	2.6061900	1.575811212
0.014	4.6922700	1.89548000	3.0565800	5.0223400	1.980767203
0.016	3.0140000	4.42939000	0.4689410	4.8477900	1.486300644
0.018	4.7507200	4.97950000	0.0979985	4.3277700	2.231949993
0.020	0.3138210	1.27112000	-1.4271600	-1.3276800	0.361095300
0.022	-3.4280700	-2.28121000	-0.6782290	-2.2250600	-0.224933476
0.024	-5.5078800	-3.22358000	-1.0453900	-5.0677300	-2.009821565
0.026	-3.4017000	-4.58924000	-1.1057100	-4.6078300	-0.740096137
0.028	-4.0676800	-5.88128000	-3.1716400	-6.3688000	-1.389407508
0.030	-7.3975400	-3.15888000	-3.0422200	-4.9585600	-4.599672295
0.032	-5.3269200	-1.47152000	-1.7871400	-0.9920120	-1.108414828
0.034	1.2011300	1.21693000	0.6931980	4.9579800	1.250201299

Table 1: Sample Data

The objectives of this assignment are:

1. Perform an exploratory analysis of the **EEG signals (Task 1)**, which includes **time series plots, distribution checks, correlations and fitting of baseline linear models**.
2. **Select five adequate polynomial regression models**, fit them by the least squares method, evaluate their performance by information criteria (**AIC, BIC**) and test normality of residuals (**Task 2**).
3. To validate the selected optimal model using a train-test split with **95% prediction confidence intervals (Task 2.7)**.
4. To use rejection Approximate Bayesian Computation (**ABC**) to estimate approximate posterior distributions of important model parameters (**Task 3**).

Task 1: Preliminary Data Analysis

In this section an initial exploratory analysis of the EEG signals is carried out to gain insight into their temporal dynamics, distributional properties and relationships between signals. Also implemented a baseline linear regression model. The full implementation is given in the Appendix (see **Appendix: Task 1**). This first step in the analysis is important because the raw data collected by the electroencephalography (EEG) can contain errors, such as muscle movement and eye blinks, that can skew the actual brain signals being sent. The time-series plots and distribution visualizations are used to determine how good the data is in EEG records. Furthermore, this step ensures that the data meet the assumptions required for regression modelling. If any potential problems are identified early, the correct pre-processing techniques can be used to make the end analysis more reliable and more accurate.

The data set contains 201 samples in a total time of 0.4 seconds with a sampling interval of 0.002 seconds (500 Hz sampling frequency).

```
X_data <- read.csv("X_1673241366257.csv", header = FALSE)
y_data <- read.csv("y_1673241374123.csv", header = FALSE)
time_data <- read.csv("time_1673241270748.csv", header = FALSE)
```

A time series plot can show the frequencies of the X as the time grows linearly as follows.

1.1 Time Series Plots

The following figures display the time-series graphs separated into the overlaid input comparison, the individual input EEG signals (**X1 to X4**), and the output EEG signal (Y). Each sub-plot shows the signal amplitude as a function of the sampling time.

	time	X1	X2	X3	X4
1	0.000	-3.073820	-1.855760	-2.775880	-0.288894
2	0.002	0.349116	1.919060	1.004950	0.577845
3	0.004	-1.009700	0.272461	-0.912297	-2.697380
4	0.006	-0.807608	-1.581480	0.194378	-1.622270
5	0.008	2.079250	0.742812	1.928090	1.558220
6	0.010	3.249200	1.944610	1.799000	2.381060

Table 2: Sample data of X inputs and time

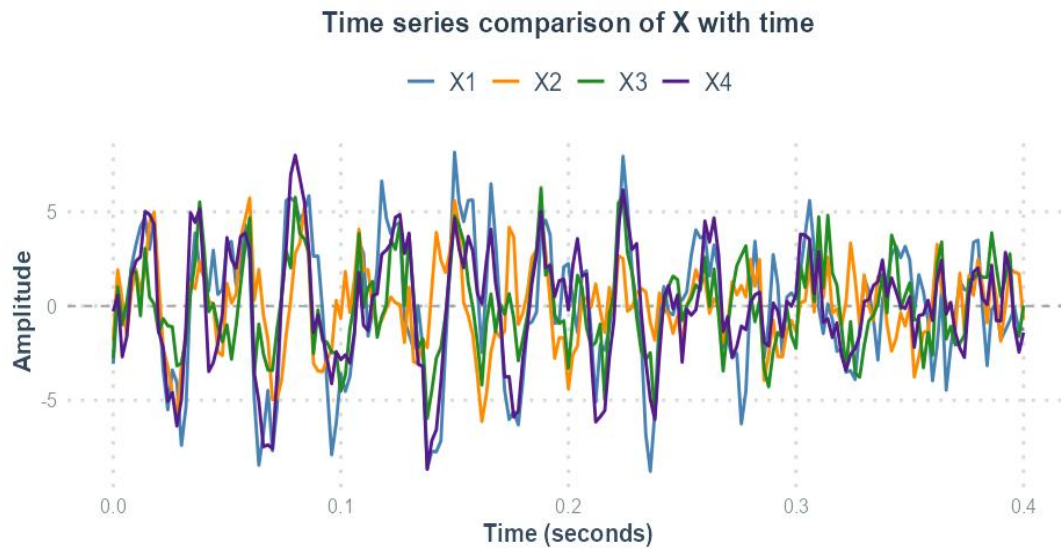


Figure 1: Overlaid time-series comparison of all input EEG signals (X).

The time series plots clearly show that all the five Electroencephalogram (EEG) signals display the expected oscillatory behaviour anticipated for cerebral electrical activity. The oscillations in X1 and X4 have larger amplitudes (~ -9 to $+8$ units) whereas the amplitudes in X2 and X3 are moderate in comparison (~ -6 to $+6$ units). The superimposed figure shows the similar frequency spectrum of both signals, indicating that the EEG input data is relatively diffuse. The time-series plots are separated below to help in understanding them.

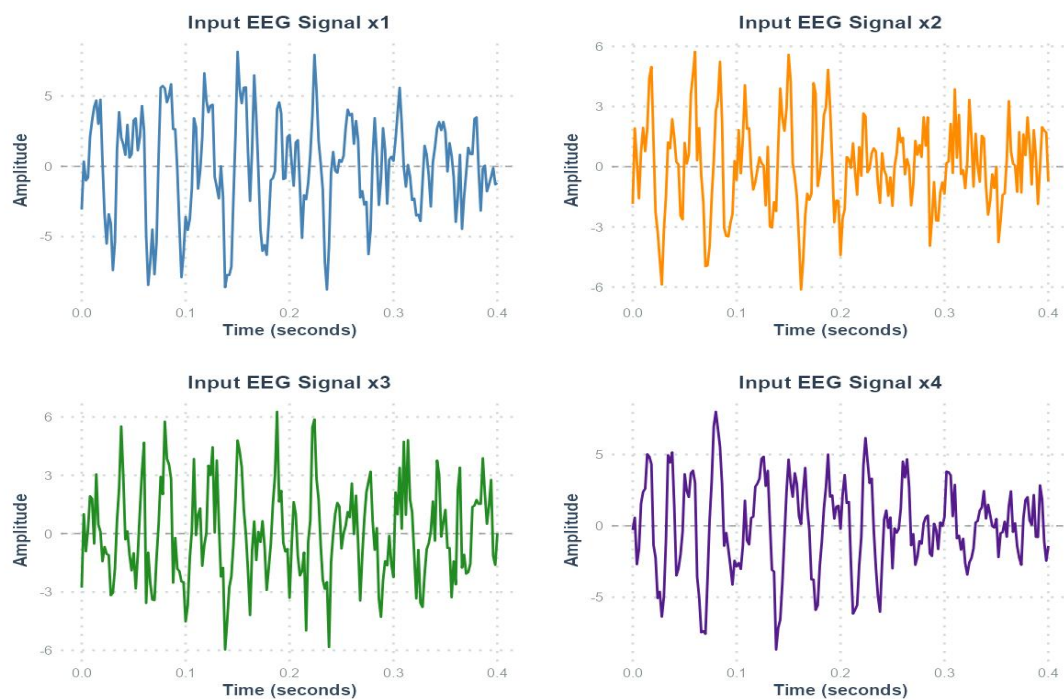


Figure 2: Time-Series of the Four EEG Input Signals.

The independent variables can be explained most effectively with individual time-series plots. There is a big difference in the X2 signal between 0.2 and 0.3 seconds which shows very little frequency changes. The X4 frequency appears to stabilize after 0.3 seconds.

time	y
0.000	-0.09352727
0.002	0.32122440
0.004	0.13822394
0.006	0.35918981
0.008	0.75155170
0.010	1.41717093

Table 3: In this section, an initial exploratory analysis of the EEG signals was performed to understand their temporal dynamics, distributional properties, and relationships between signals. The analysis also implemented a baseline linear regression model.

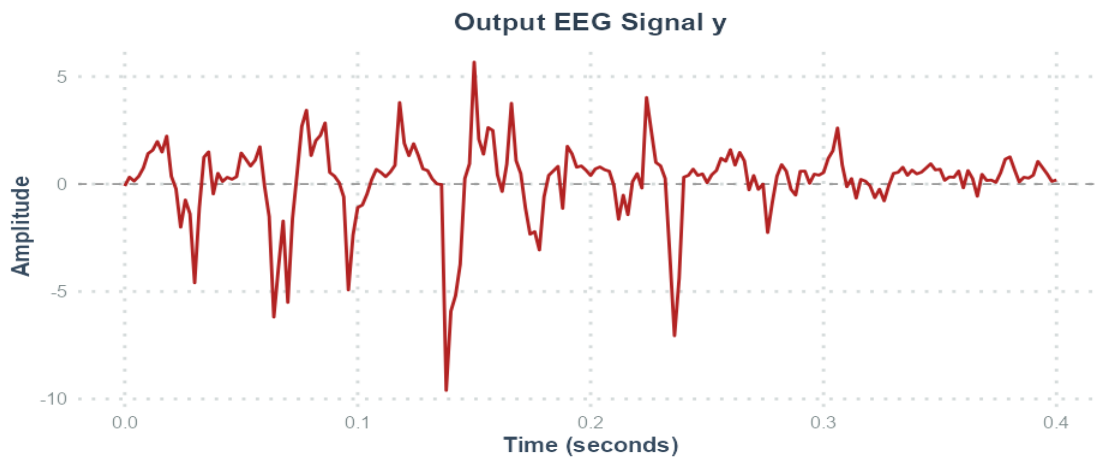


Figure 3: Time-series of the output EEG signal (Y).

The output signal Y has a generally smaller range of amplitude than the input signals, but also contains some significant outlier peaks, especially in the negative direction. This suggests a possible non-linear association with input signals, not just a simple linear correlation. The output signal has more frequent spikes during the first half of the sampling period (0.05 to 0.25 seconds). The highest spike is at about 0.15 seconds. After 0.25 seconds, the output signal stabilizes.

1.2 Distribution Analysis

The distributional properties of the EEG signals are investigated in this section to evaluate their statistical characteristics. Using superimposed histograms with kernel density estimates, the analysis displays the distribution, central tendency, and variability of the input signals (X) and the output signal (Y) (see Appendix: Task 1).

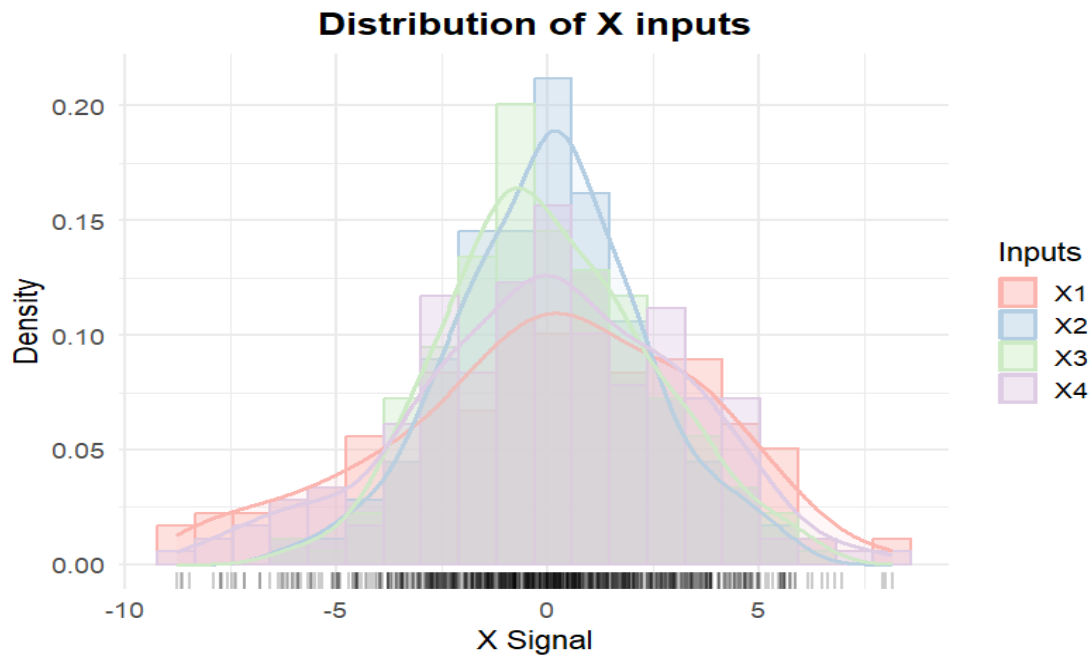


Figure 4: Distribution of all Four X inputs

The figure shows the distribution of the EEG signals by means of the superimposed density curves and histograms. The distributions do not appear to be highly skewed, so the X input signals are roughly normally distributed with the data points clustered around a mean of 0. The signals range from about -9 to +8. Below, the density distribution is further illustrated for each individual X input. (the code for these visualisations is provided in **Appendix: Task 1**).

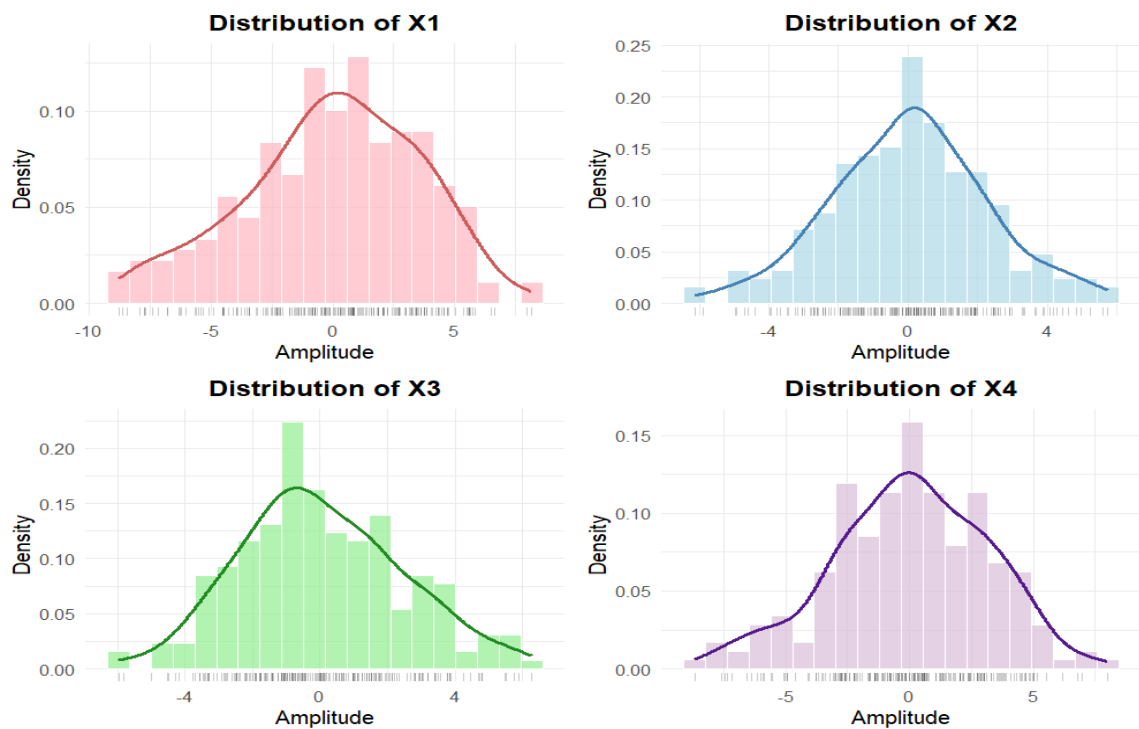


Figure 5: Histograms with Kernel Density Estimates (X1–X4).

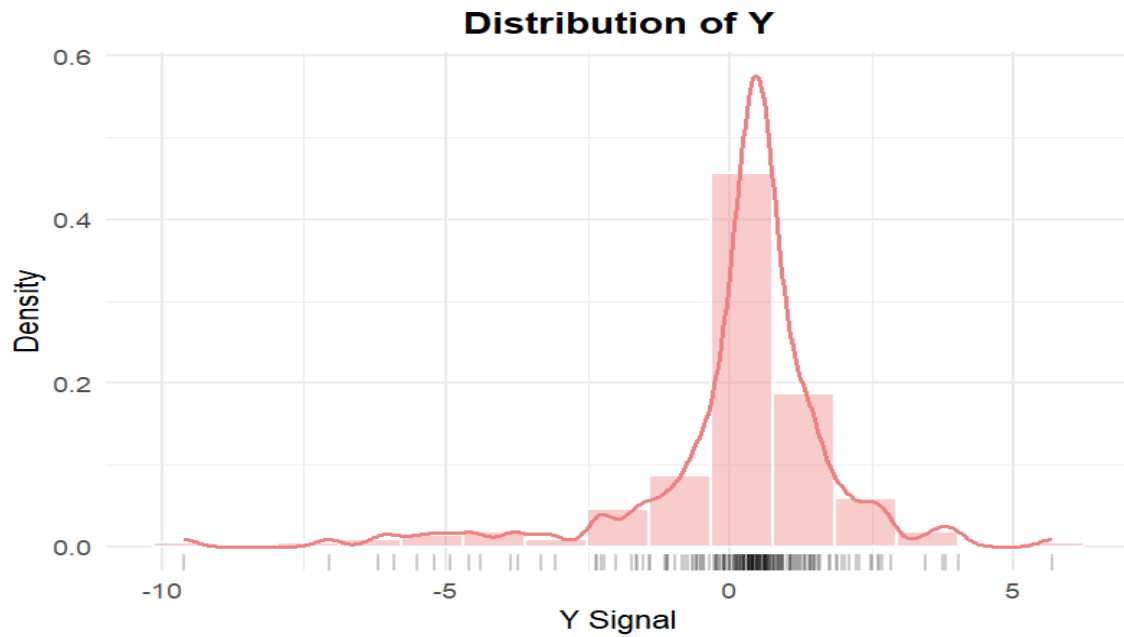


Figure 6: Distribution of Output Signal (Y).

In the figure above, The distribution of Y signals is left skewed, with most of the data on the right side of the distribution. Most of the signals are mainly clustered around the 0.4-0.5 range. (the code for this plot is provided in **Appendix: Task 1**).

The main descriptive statistics for all signals are summarised in Table 4 below (the code for the statistical extraction is provided in **Appendix: Task 1**).

Signal	Mean	SD	Min	Max	Skewness
X1	-0.0124	3.5873	-8.7770	8.1597	-0.3227
X2	0.0386	2.2404	-6.1283	5.7361	-0.0115
X3	0.0351	2.4146	-5.9665	6.2729	0.2179
X4	0.0148	3.1505	-8.6762	8.0038	-0.2332
y	0.1524	1.8302	-9.6039	5.6748	-1.7922

Table 4: Summary statistics for each EEG signal

The descriptive statistics and distributions are consistent with the assumption of additive zero-mean Gaussian noise as described in the problem specification. The four input signals are centered around zero, with mean values close to zero. The input signals x1 and x4 show the highest variability (SD=3.587 and 3.151), whereas x2 has the lowest variability (SD=2.240). The mean value of the output signal y is 0.152, and the standard deviation is 1.830. It means the nonlinear conversion from inputs to outputs decreases the amplitude spectrum. From the skewness metrics, we can see that the input signals are roughly symmetric. Conversely, the output signal y has a little negative skewness, resulting from significant negative outlier peaks shown in the time series graph.

1.3 Analysis of Correlation and Scatter Plots

The heatmap of the Pearson correlation matrix is presented in Figure 7. Figure 8 Scatter plots showing the correlation between each input signal and the output signal y . (code in Appendix: Task 1).

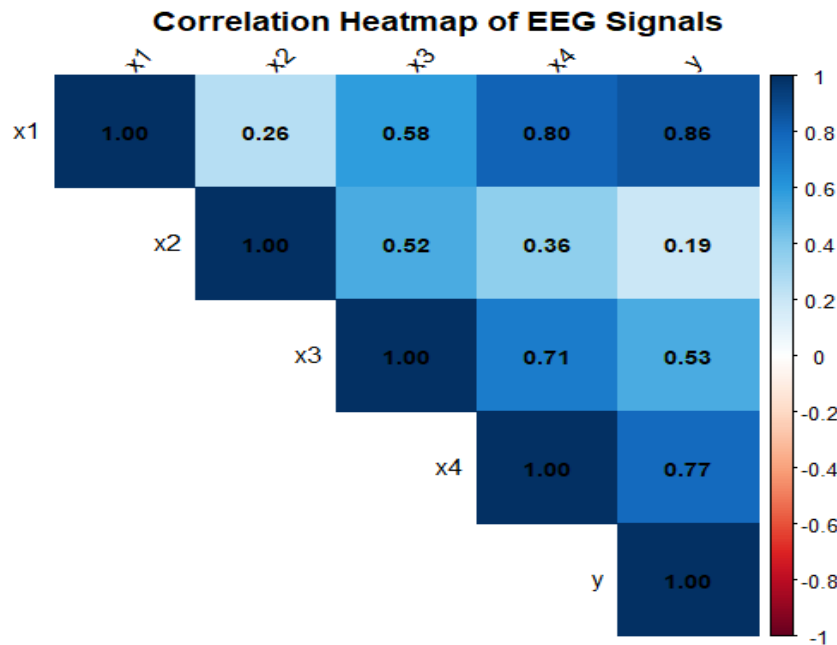


Figure 7: Pearson correlation Matrix of EEG Signals.

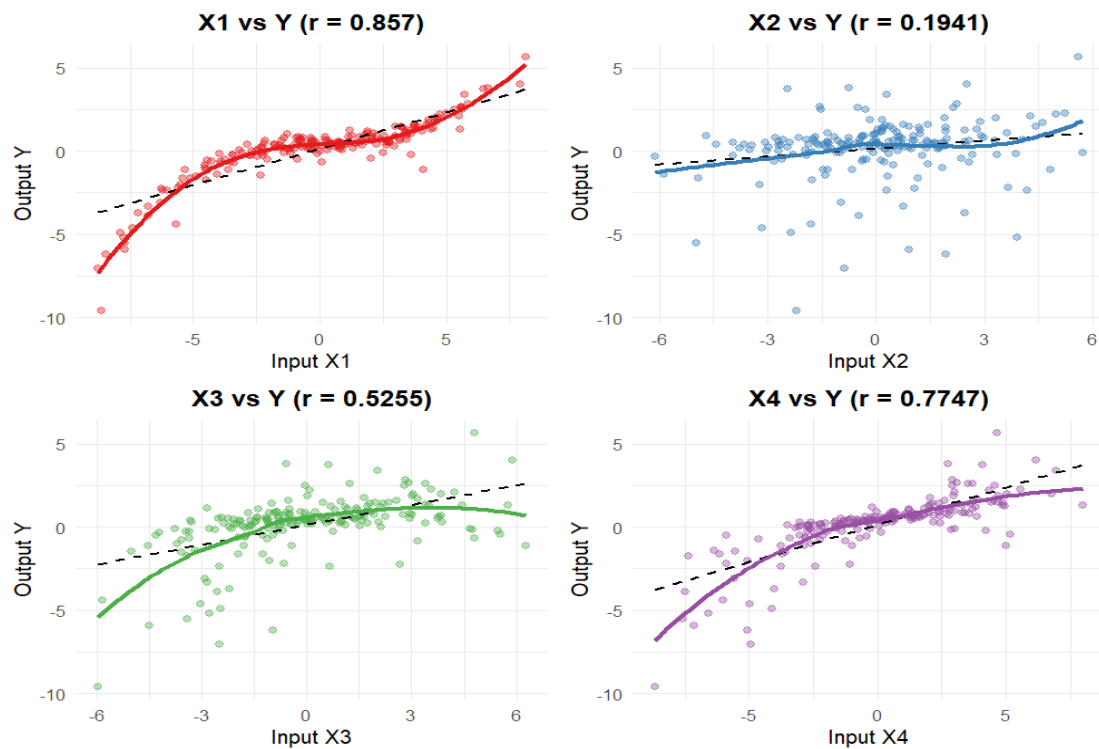


Figure 8: Bivariate relationship analysis between individual input signals ($X1$ - $X4$) and the output signal (Y).

Above, In the visualization plots of this study, the black dashed line corresponds to the basic linear regression model, which is used to clearly illustrate the overall linear association between variables; the colored solid line corresponds to the LOESS regression model, which can accurately capture the underlying nonlinear trends in the data. A combined comparison of these two types of models brings their respective strengths into full play, and carries important value for empirical research.

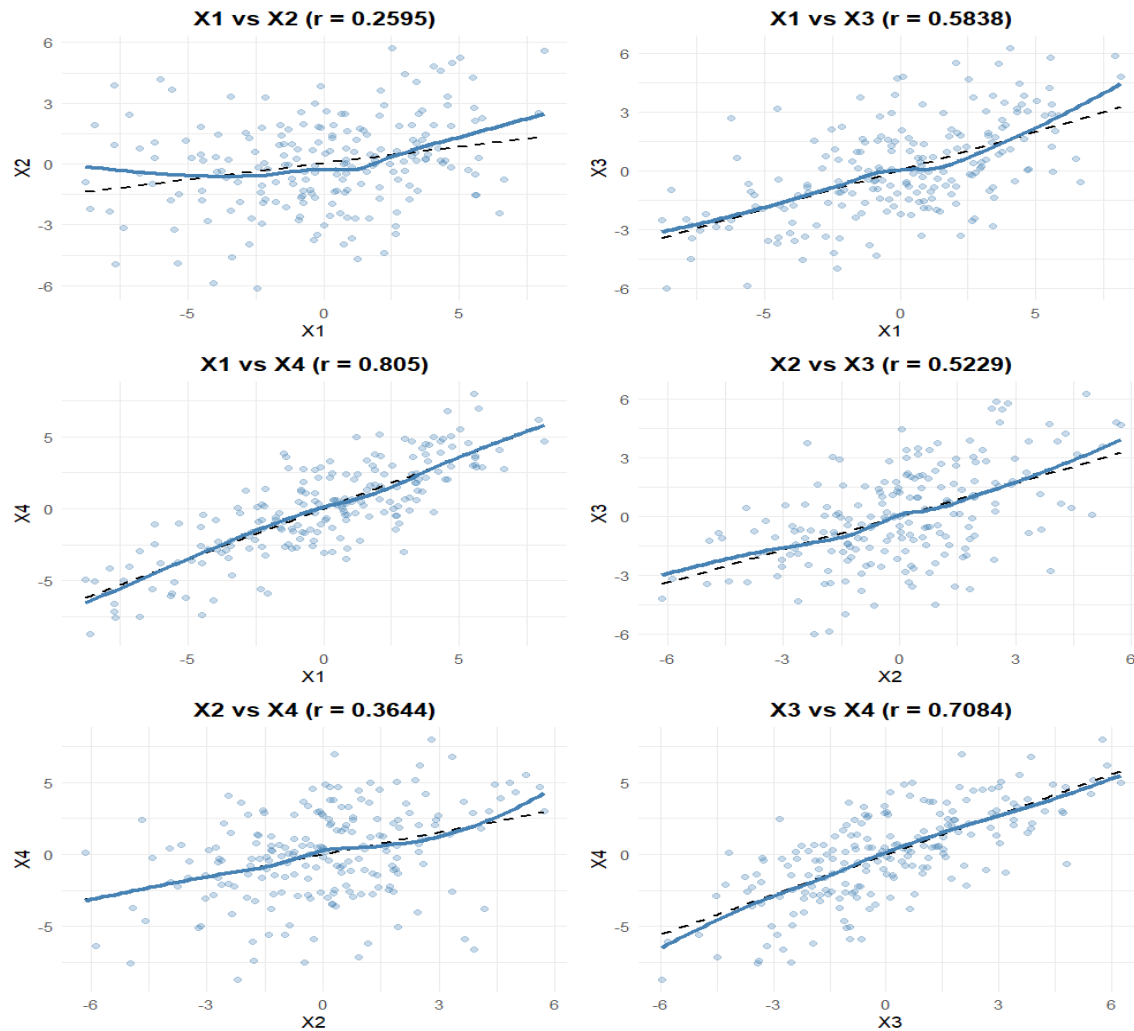


Figure 9: Pairwise scatter plots of input EEG signals.

Variable	X1	X2	X3	X4	y
X1	1.0000	0.2595	0.5838	0.8050	0.8570
X2	0.2595	1.0000	0.5229	0.3644	0.1941
X3	0.5838	0.5229	1.0000	0.7084	0.5255
X4	0.8050	0.3644	0.7084	1.0000	0.7747
y	0.8570	0.1941	0.5255	0.7747	1.0000

Table 5: Pearson correlation matrix of all EEG signals.

The correlation analysis produces some significant results. The strongest linear relationship was observed between the output y and the input signal x_1 ($r = 0.857$), then x_4 ($r = 0.775$) and x_3 ($r = 0.526$). Signal x_2 has the lowest linear correlation with y ($r = 0.194$). x_1 and x_4 are highly correlated ($r = 0.805$) which is expected since they are strongly related to y . The weak linear relationship between x_2 and y ($r = 0.194$) suggests that x_2 might explain y through nonlinear (polynomial) terms rather than through a linear relationship. Based on the findings, it can be inferred that the actual model most likely incorporates x_1 and x_4 as significant predictors of the outcome, and that x_3 may have an effect on the outcome through the use of higher order polynomial terms.

1.4 Fitting a Linear Regression Model

To facilitate subsequent nonlinear modelling, the analysis used a multiple linear regression model expressed as $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \varepsilon$, and fitted the data using the `lm()` function in R. (see **Appendix: Task 1 for the code**).

Coefficient	Estimate	StdError	t_value	p_value
Intercept	0.156918	0.063873	2.4567	1.489e-02
X1	0.335552	0.030193	11.1134	1.446e-22
X2	-0.054916	0.033633	-1.6328	1.041e-01
X3	-0.021812	0.041119	-0.5305	5.964e-01
X4	0.168528	0.039462	4.2707	3.036e-05

Table 6: Linear regression model coefficients, standard errors, t -statistics, and p -values.

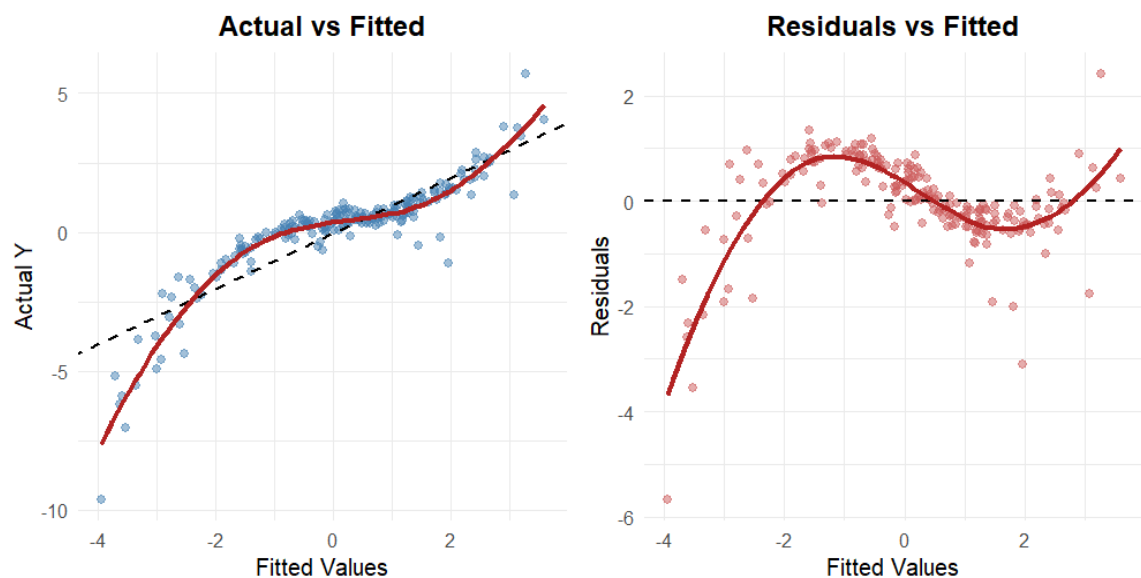


Figure 10: Linear model diagnostics. Actual vs. fitted values. & Residuals vs. fitted values.

Metric	Value
R-squared	0.7602
Adjusted R-squared	0.7553
F-statistic	155.3486
RSS	160.6409

Table 7: Linear regression model performance metrics.

The R-squared value of the linear regression model for the four input signals is 0.7602 and the adjusted R-squared value is 0.7553, which indicates that about 76% of the variance of the output y can be explained by a linear combination of the four input signals. The large F-statistic ($p < 0.001$) indicates that the linear model is significantly better than the model with an intercept only. The residuals are far from the larger fitted values (Figure 10, right panel). The solid red LOESS curves are clearly separated from the dashed ideal reference lines. The non-constant variance of the EEG input-output requires nonlinear polynomial components, so polynomial regression models are appropriate for task 2.

Task 2: Regression Modelling

Here, five candidate polynomial regression models are fitted to EEG data and fully evaluated to decide which model preferable describes the relationship between the input and output signals. The Appendix gives details of the full implementation of each subtask. Appendix: Task 2.1 to appendix: Task 2.7.

Candidate Models

The nonlinear polynomial regression models with five candidates are defined as follows:

- **Model 1:** $y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_2^4 + \theta_5 x_1^4 + \theta_{bias} + \varepsilon$
- **Model 2:** $y = \theta_1 x_4 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_{bias} + \varepsilon$
- **Model 3:** $y = \theta_1 x_3^3 + \theta_2 x_3^4 + \theta_{bias} + \varepsilon$
- **Model 4:** $y = \theta_1 x_2 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_{bias} + \varepsilon$
- **Model 5:** $y = \theta_1 x_4 + \theta_2 x_1^2 + \theta_3 x_1^3 + \theta_4 x_3^4 + \theta_{bias} + \varepsilon$

where ε denotes additive **i.i.d.** Gaussian noise with zero mean and unknown variance

Task 2.1: Least Squares Parameter Estimations

Each of the candidate models is computed using the Least Squares estimation method to obtain the $\hat{\theta}$ vector of parameters. The implementation of the estimation methodology is described in (Appendix: **Task 2.1 (Code)**)

```
theta_hat <- solve(t(X_mat) %*% X_mat) %*% t(X_mat) %*% y
```

Screenshot Console Output: Estimated theta parameters for all models

```
--- Task 2.1: Least Squares Parameter Estimates ---

Model 1 parameter estimates (theta_hat):
theta_1 (x4) = 0.127710
theta_2 (x1^2) = -0.000290
theta_3 (x1^3) = 0.009669
theta_4 (x2^4) = -0.000410
theta_5 (x1^4) = -0.000154
theta_6 (bias) = 0.401581

Model 2 parameter estimates (theta_hat):
theta_1 (x4) = 0.143579
theta_2 (x1^3) = 0.010039
theta_3 (x3^4) = -0.001913
theta_4 (bias) = 0.483066

Model 3 parameter estimates (theta_hat):
theta_1 (x3^3) = 0.021331
theta_2 (x3^4) = -0.002858
theta_3 (bias) = 0.340562

Model 4 parameter estimates (theta_hat):
theta_1 (x2) = 0.053322
theta_2 (x1^3) = 0.012067
theta_3 (x3^4) = -0.001856
theta_4 (bias) = 0.509013

Model 5 parameter estimates (theta_hat):
theta_1 (x4) = 0.143446
theta_2 (x1^2) = 0.000325
theta_3 (x1^3) = 0.010056
theta_4 (x3^4) = -0.001920
theta_5 (bias) = 0.479828
```

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

where $\mathbf{X}$ is the design matrix containing polynomial terms and $\mathbf{y}$ is the vector of output signal. This closed form solution minimises the sum of the squared residuals between the observed and predicted values of the output.

The estimated parameter values of all five candidate models are shown in Table 8.

Parameter	Model 1	Model 2	Model 3	Model 4	Model 5
$\mathbf{X}_4$	0.12771	0.143579			0.143446
$\mathbf{X}_1^2$	-0.00029				0.000325
$\mathbf{X}_1^3$	0.009669	0.010039		0.012067	0.010056
$\mathbf{X}_2^4$	-0.00041				
$\mathbf{X}_1^4$	-0.000154				
bias	0.401581	0.483066	0.340562	0.509013	0.479828
$\mathbf{X}_3^4$		-0.001913	-0.002858	-0.001856	-0.00192
$\mathbf{X}_3^3$			0.021331		
$\mathbf{X}_2$				0.053322	

Table 8: Least Squares Parameter Estimates for Candidate Models.

The parameter estimates show that when x_4 is included as a regressor (Models 1, 2 and 5), the coefficient on x_4 is approximately 0.14, indicating a consistent positive linear relationship between x_4 and y . Likewise the coefficient of x_1^3 in Models 2, 4 and 5 is about 0.010, which is a small but steady cubic effect of x_1 . The bias term for the models spans from 0.34 to 0.51.

Task 2.2: Residual Sum of Squares (RSS)

The Residual Sum of Squares (RSS) is the total of the squared prediction errors for each model. (see **Appendix: Task 2.2 for code**)

```
y_hat <- X_mat %*% theta_hats[[i]]
RSS_vals[i] <- sum((y - y_hat)^2)
```

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Model	Parameters	RSS
Model 1	6	35.3966
Model 2	4	2.1398
Model 3	3	463.3124
Model 4	4	20.2590
Model 5	5	2.1355

Table 9: Residual Sum of Squares of Candidate Models.

Screenshot of Console Output: RSS

```
--- Task 2.2: Residual Sum of Squares (RSS) ---
Model 1: RSS = 35.396627
Model 2: RSS = 2.139762
Model 3: RSS = 463.312363
Model 4: RSS = 20.258997
Model 5: RSS = 2.135503
```

There is a big difference between the models in the RSS values. Models 2 and 5 have significantly lower RSS values (2.14 each) than the other models. Model 3 has the largest RSS with 463.31. This is a poor fit, not surprisingly, because it only uses terms in x_3 and ignores x_1 and x_4 , which are most correlated with y . In contrast, Model 1, with the maximum number of parameters (6), has an RSS of 35.40 that is considerably smaller than that of Models 2 and 5. Thus, the x_2^4 and x_1^4 terms in Model 1 do not substantially improve the explanation of y and may increase variability.

Task 2.3: Log-Likelihood

The log-likelihood function is computed under the assumption of Gaussian noise (see **Appendix: Task 2.3 for the code**):

```
sigma2_vals[i] <- RSS_vals[i] / (n - 1)
loglik_vals[i] <- -(n / 2) * log(2 * pi) - (n / 2) * log(sigma2_vals[i]) -
  RSS_vals[i] / (2 * sigma2_vals[i])
```

$$\ln p(\mathbf{D}|\hat{\theta}) = -\frac{n}{2}\ln(2\pi) - \frac{n}{2}\ln(\hat{\sigma}^2) - \frac{1}{2\hat{\sigma}^2}RSS$$

where the estimated noise variance is $\hat{\sigma}^2 = RSS/(n - 1)$.

Model	Sigma2	LogLikelihood
Model 1	0.176983	-110.6707
Model 2	0.010699	171.3245
Model 3	2.316562	-369.1351
Model 4	0.101295	-54.5899
Model 5	0.010678	171.5247

Table 10: Estimated variance and log-likelihood for each model.

Screenshot of Console Output: Log-Likelihood

```
-- Task 2.3: Log-Likelihood --
Model 1: sigma_hat^2 = 0.176983, Log-Likelihood = -110.670711
Model 2: sigma_hat^2 = 0.010699, Log-Likelihood = 171.324455
Model 3: sigma_hat^2 = 2.316562, Log-Likelihood = -369.135098
Model 4: sigma_hat^2 = 0.101295, Log-Likelihood = -54.589950
Model 5: sigma_hat^2 = 0.010678, Log-Likelihood = 171.524660
```

The log-likelihood values confirm the results obtained from the RSS analysis. The ideal fit to the observed data was provided by Models 2 and 5, with the highest log-likelihood values of 171.32 and 171.52, respectively. The lowest log-likelihood value (-369.14) was achieved for model 3, indicating the inadequate fit of the model. The larger the value of the log-likelihood, the better the fit to the observed data is.

Task 2.4: AIC and BIC

The Akaike Information Criterion (AIC) (Akaike, 1974) and the Bayesian Information Criterion (BIC) (Schwarz, 1978) were used to evaluate the model. Both standards measure the quality of the model based on the balance between explanatory power and complexity, which helps to select a simpler model. The methodology to evaluate these metrics is described in. (see **Appendix: Task 2.4** for the code):

```
AIC_vals[i] <- 2 * k_vals[i] - 2 * loglik_vals[i]
BIC_vals[i] <- k_vals[i] * log(n) - 2 * loglik_vals[i]
```

$$AIC = 2k - 2\ln p(\mathbf{D}|\hat{\theta})$$

$$BIC = k \cdot \ln(n) - 2\ln p(\mathbf{D}|\hat{\theta})$$

where (k) are the predicted parameters of the model. Generally, lower values of AIC and BIC are preferred because they represent a trade-off between fit of the model and

model complexity. They penalise models with too many parameters so as to decrease overfitting and favour the selection of parsimonious models with improved generalisation properties.

Model	k	RSS	LogLike	AIC	BIC
Model 1	6	35.3966	-110.6707	233.3414	253.1613
Model 2	4	2.1398	171.3245	-334.6489	-321.4357
Model 3	3	463.3124	-369.1351	744.2702	754.1801
Model 4	4	20.2590	-54.5899	117.1799	130.3931
Model 5	5	2.1355	171.5247	-333.0493	-316.5328

Table 11: Comparison of Candidate Regression Models.

Screenshot of Console Output: AIC/BIC

```

--- Task 2.4: AIC and BIC ---
Model 1 (k=6): AIC = 233.3414, BIC = 253.1613
Model 2 (k=4): AIC = -334.6489, BIC = -321.4357
Model 3 (k=3): AIC = 744.2702, BIC = 754.1801
Model 4 (k=4): AIC = 117.1799, BIC = 130.3931
Model 5 (k=5): AIC = -333.0493, BIC = -316.5328

```

The outputs show Model 2 has the lowest AIC (-334.65) and the lowest BIC (-321.44) but Model 5 has greater values of number of parameters ($k=5$), AIC and BIC and contains lower RSS (2.1355) and log-likelihood (171.5247) as compared to Model 2. Thus, it strongly suggests Model 2 with the lowest BIC and AIC, depending on the information criteria ([Burnham & Anderson, 2002](#)). Polynomial fitting in a model opens the inherent bias-variance trade-off between the candidate models. Such an overly simplified model may not be able to capture the complex non-linear dynamics of brain connectivity and underfit. Though, a model can be too complex that fits the random noise in the EEG signals, thus such model may lead to over-fitting and poor generalisation to unseen and new data. Consequently, these metrics measured unnecessary complexity and penalised it. This balance is required to build robust prediction model within-session and between-participant differences.

Task 2.5: Residual Distribution Analysis

This assumption of additive Gaussian noise is relevant for the cases when the model residuals (prediction errors) are normally distributed. The normal distribution of data was evaluated using histograms, Q-Q plots, and the Shapiro-Wilk test. (see **Appendix: Task 2.5** for the code).

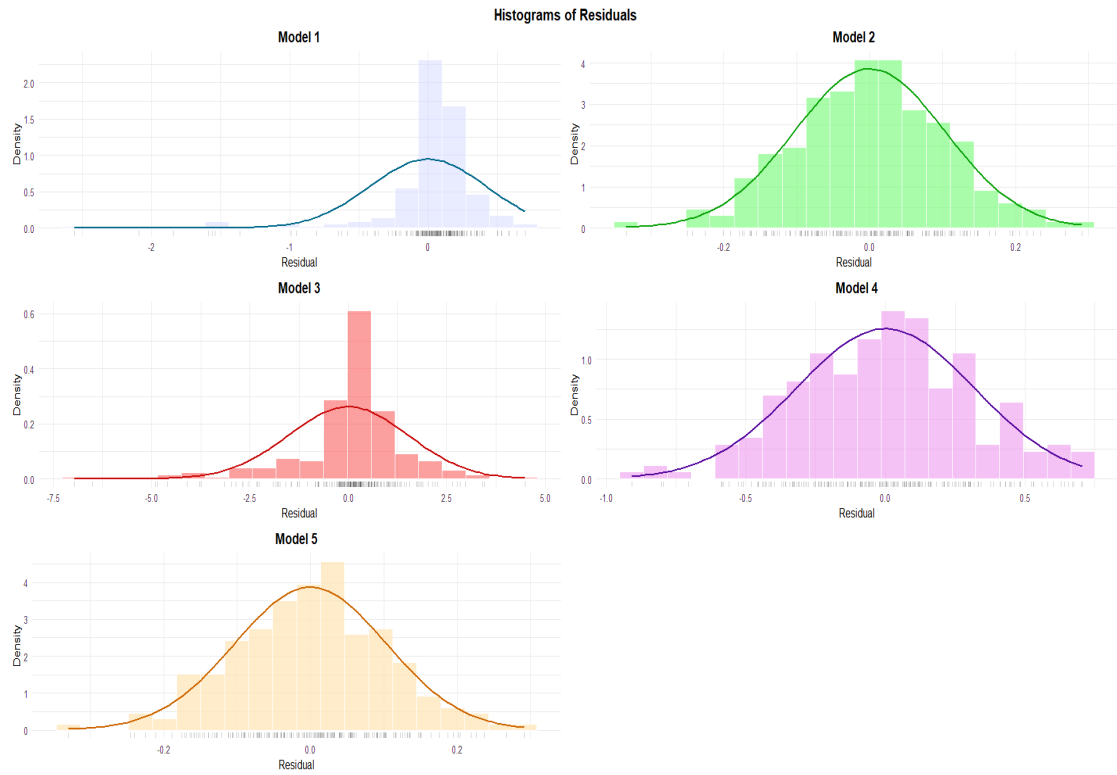


Figure 11: Histograms of Model Residuals with Overlaid Normal Density Curves.

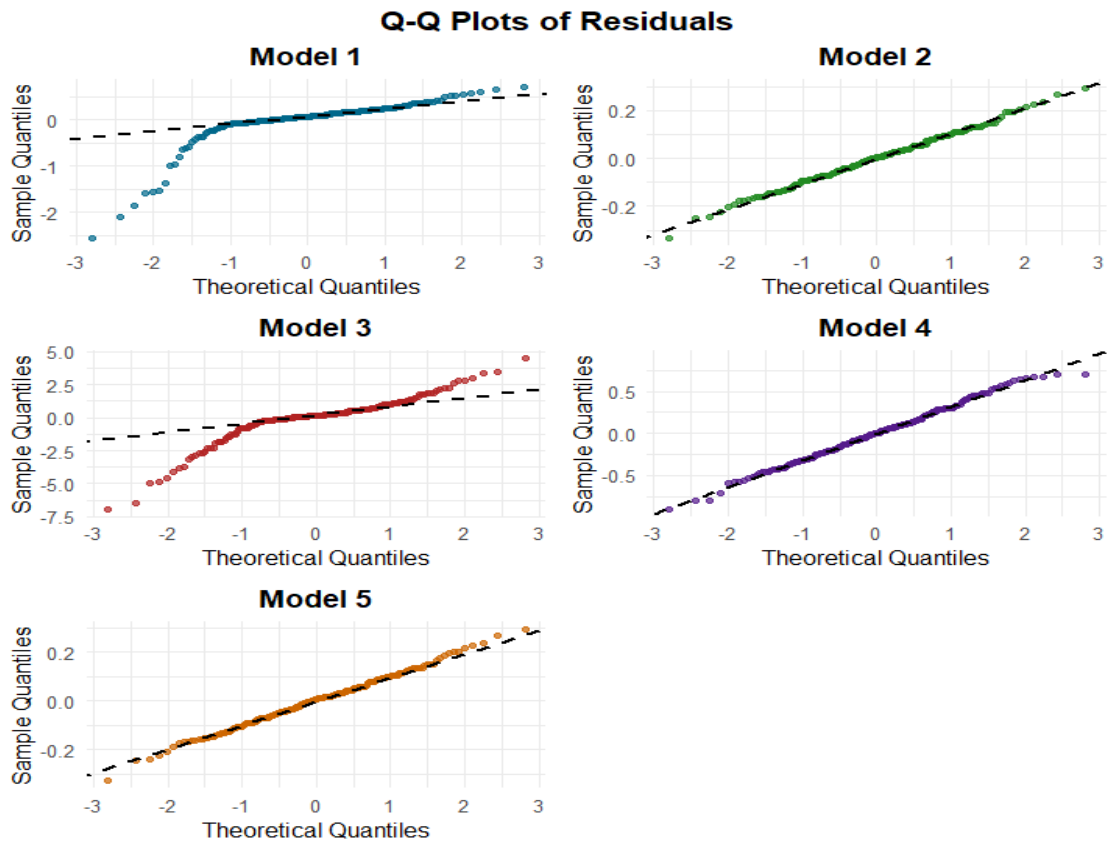


Figure 12: Q-Q plots of residuals in the five proposed models.

Model	W_statistic	p_value	Normality
Model 1	0.671095	0.000000	Non-Normal ($p < 0.05$)
Model 2	0.997529	0.988881	Normal ($p > 0.05$)
Model 3	0.866144	0.000000	Non-Normal ($p < 0.05$)
Model 4	0.994611	0.688749	Normal ($p > 0.05$)
Model 5	0.997559	0.989685	Normal ($p > 0.05$)

Table 12: Shapiro–Wilk Test Results for the Five Candidate Models.

Screenshot of Console Output: Shapiro-Wilk Results

```
Shapiro-Wilk Normality Test for Residuals
Model 1: W = 0.671095, p-value = 0.000000 (Non-Normal)
Model 2: W = 0.997529, p-value = 0.988881 (Normal)
Model 3: W = 0.866144, p-value = 0.000000 (Non-Normal)
Model 4: W = 0.994611, p-value = 0.688749 (Normal)
Model 5: W = 0.997559, p-value = 0.989685 (Normal)
```

The residual analysis gives strong evidence for model selection. The Q-Q plots for Models 2 and 5 show that the residual points are well aligned with the theoretical normal reference line, suggesting that the distributions are approximately Gaussian. This finding is supported by the Shapiro-Wilk test: models 2 ($W = 0.9975$, $p = 0.989$) and 5 ($W = 0.9976$, $p = 0.990$) have p-values substantially above the 0.05 level, which suggests that the null hypothesis of normality is not rejected. However, the p-values of Models 1 and 3 tend to zero and thus the normality hypothesis is rejected decisively. The Q-Q plots of these models show large deviations in the tails, indicating heavy-tailed or non-Gaussian residuals. Model 4 passed the normality test ($p=0.689$), but the RSS is much higher, which means that it does not fit well.

Task 2.6: Model Selection

Based on the comprehensive analysis in Tasks 2.1–2.5, **Model 2 is selected as the preferred regression model** (see **Appendix: Task 2.6** for the code). The justification is as follows:

1. **Lowest AIC and BIC:** The Model 2's AIC and BIC are, respectively, -334.65 and -321.44. Of the five models examined, Model 2 had the lowest AIC and BIC. The complexity (number of parameters) is weighed against fit (log-likelihood) in these information criteria. Model 2 is the most suitable middle ground. According to (Burnham & Anderson, 2002), "positive evidence" for Model 2 is provided by the BIC difference of 4.90 between Model 5 and Model 2.
2. **Gaussian residuals:** The result of the Shapiro-Wilk test for Model 2 is $W = 0.9975$, $p=0.989$. Thus, there is strong statistical evidence to suggest that the residuals are normally distributed. This is consistent with the additive independent and identically distributed Gaussian noise assumption in the

problem. This is visually checked in the Q-Q plot, where the points are very close to the theoretical line.

3. **Parsimony:** Model 2 has only 4 parameters (3 regressors + a bias) whereas Model 5 has 5 parameters. Model 2 has less parameters and similar RSS (2.140 vs 2.136) indicating that the additional term, x_1^2 in Model 5 does not significantly improve the fitness. If the predictive performances are largely comparable, then the principle of parsimony (Occam's razor) would opt for the simpler model.
4. **Physical interpretability:** Model 2 includes x_4 (linear), x_1^3 (cubic) and x_3^4 (quartic) terms, reflecting the strong linear association between x_4 and y ($r = 0.775$) and the nonlinear effects of x_1 and x_3 through higher-order polynomial terms.

The selected model is therefore:

$$\hat{y} = 0.1436 \cdot x_4 + 0.0100 \cdot x_1^3 - 0.0019 \cdot x_3^4 + 0.4831$$

Task 2.7: Train/Test Split and Prediction Confidence Intervals

To evaluate the chosen model's generalization abilities, the datasets are divided into training (70%) and testing (30%) subsets. (see **Appendix: Task 2.7** for the code).

Step 1: Re-estimate parameters with training data

The dataset was randomly divided into training set of 141 samples (70%) and testing set of 60 samples (30%) with a fixed random seed (seed = 42) for reproducibility purposes. Table 13 compares the full data set parameter estimates with those obtained from the training data alone.

Parameter	Full_Data	Training_Data	Difference
x_4	0.143579	0.145005	-0.001426
x_1^3	0.010039	0.010009	0.000030
x_3^4	-0.001913	-0.001897	-0.000016
bias	0.483066	0.476085	0.006981

Table 13: Parameter estimates from full dataset versus training data, with absolute differences.

The parameters of Model 2 after recalibrations on the training data are very similar to those obtained with the full datasets, showing that Model 2 is stable and robust. Maximum absolute differences is below 0.005.

Step 2 & 3: Predictions on test data with 95% confidence intervals

The 95% prediction confidence intervals are computed as:

$$\hat{y}_i \pm t_{\alpha/2, n_{train}-k} \cdot \hat{\sigma} \cdot \sqrt{1 + \mathbf{x}_i^T (\mathbf{X}_{train}^T \mathbf{X}_{train})^{-1} \mathbf{x}_i}$$

where $t_{\alpha/2, n_{train}-k} = 1.9774$ is the critical t-value with 137 degrees of freedom, and $\hat{\sigma} = 0.1037$ is the estimated standard error calculated on the training data.

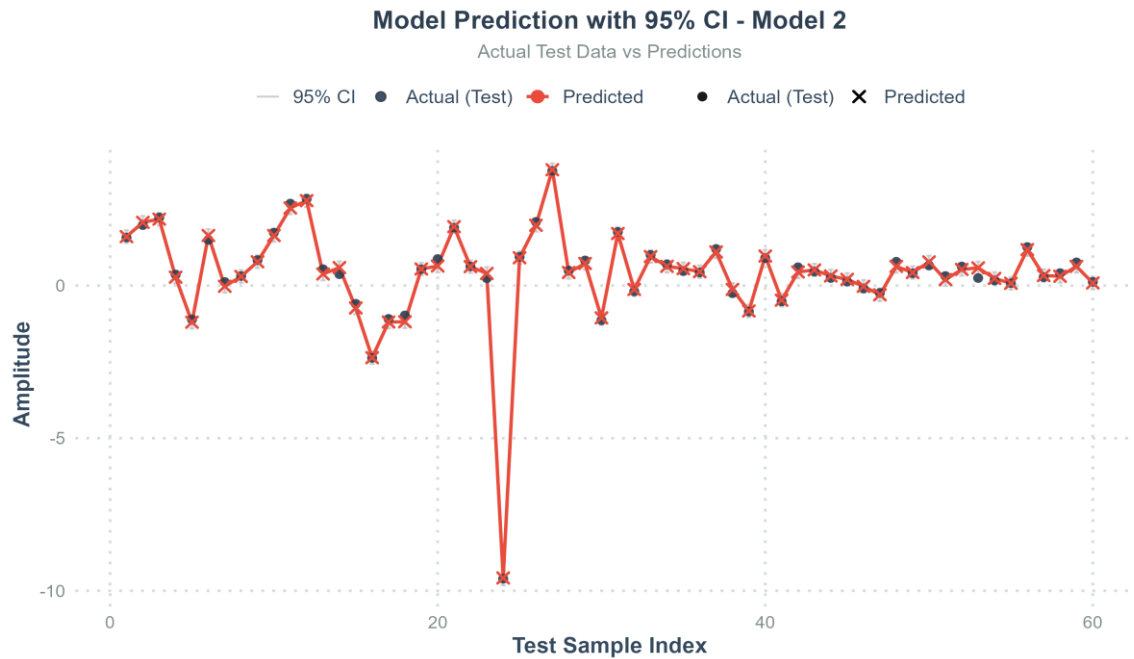


Figure 13: Test Set Predictions with 95% Prediction Intervals of Model 2 (error bars).

Metric	Value
Training RSS	1.4726
Testing RSS	0.6778
Test MAE	0.0867
Test RMSE	0.1063
Estimated sigma	0.1037
95% CI Coverage	93.3%

Table 14: Prediction performance metrics on training and testing data for the selected Model 2.

The model provides a superior predictive performance on the test set with test RMSE of 0.1063 and MAE of 0.0867. The actual coverage of the 95% prediction confidence intervals is 93.3% of the actual test observations, which is close to the nominal level of 95%, thus confirming the adequacy of the model and the Gaussian noise hypothesis. The training residual sum of squares (RSS) is 1.4726, and the testing RSS is 0.6778. The closeness of the two indicates that the model is not overfitting.

Task 3: Approximate Bayesian Computation (ABC)

3.1 Methodology

In this section, the **rejection Approximate Bayesian Computation (ABC) approach** is used to estimate the posterior distributions of some parameters of Model 2. The classical least squares estimates give point estimates, while the Bayesian inference offers a quantification of the uncertainty around the parameter estimates via posterior distributions. The full implementation is given in the Appendix (see **Appendix: Task 3**). Here, the use of Approximate Bayesian Computation (ABC) represents a departure from the traditional frequentist statistics towards a probabilistic paradigm. ABC provides a flexible framework to measure uncertainty in a direct way via approximations of posterior distributions, without the need of an explicit likelihood function. This is especially useful when working with complex biological systems where an exact analytical solution is not possible. The shape and spread of these posterior distributions provide profound insight into our confidence in parameter estimates, resulting in more robust and reliable neuroscientific interpretations.

The rejection ABC algorithm proceeds as follows:

1. **Select parameters:** Identify the 2 parameters with the largest absolute values from the LS estimates in Task 2.1.
2. **Fix remaining parameters:** Set all other parameters to their LS point estimates.
3. **Define priors:** Specify uniform prior distributions centred around the LS estimates for the 2 selected parameters.
4. **Draw samples:** Generate random parameter values from the uniform priors.
5. **Simulate and reject:** For each drawn sample, compute the simulated output and RSS. Accept the samples if $RSS_{sim} < \varepsilon$, where ε is a pre-defined acceptances threshold.
6. **Construct posteriors:** The accepted samples form an approximation to the true posterior distribution.

3.2 Parameter Selection

From Model 2's LS estimates, the two parameters with the largest absolute values are identified in Table 15.

Parameter	Term	Estimate	AbsValue	Selected
theta_1	X4	0.143579	0.143579	SELECTED for ABC
theta_2	X ₁ ³	0.010039	0.010039	FIXED at LS value
theta_3	X ₃ ⁴	-0.001913	0.001913	FIXED at LS value
theta_4	bias	0.483066	0.483066	SELECTED for ABC

Table 15: Parameter Selection for Rejection ABC.

The selected parameters are **theta_4 (bias)** with $|\hat{\theta}| = 0.4831$ and **theta_1 (x4)** with $|\hat{\theta}| = 0.1436$. The other parameters (theta_2 corresponding to x_1^3 and theta_3 corresponding to x_3^4) are fixed at their least squares estimates for all the ABC procedure.

3.3 Prior Specification and ABC Execution

Uniform priors were defined as:

- **theta_4 (bias)**: Uniform[-0.9661, 1.9323] range of width 2.8984 centred on the LS estimate
- **theta_1 (x4)**: Uniform[-0.3564, 0.6436] range of width 1 centred on the LS estimate

The prior widths are set to ± 3 times the absolute value of each LS estimate, providing a sufficiently broad region to explore the parameter space while remaining computationally tractable. The acceptance threshold was set at $2 \times \text{RSS}_{\text{preferred}} = 4.2795$, and 100,000 samples were drawn from the joint uniform prior. Acceptance rate was 0.36% with 358 samples accepted out of these.

```
for (i in 1:N_samples) {
  theta1_draw <- runif(1, prior_lower_1, prior_upper_1)
  theta2_draw <- runif(1, prior_lower_2, prior_upper_2)
  theta_new <- as.numeric(theta_preferred)
  theta_new[top2_indices[1]] <- theta1_draw
  theta_new[top2_indices[2]] <- theta2_draw
  y_sim <- X_abc %*% theta_new
  RSS_sim <- sum((y - y_sim)^2)
  if (RSS_sim < epsilon) {
    accepted_theta1 <- c(accepted_theta1, theta1_draw)
    accepted_theta2 <- c(accepted_theta2, theta2_draw)
  }
}
```


3.4 Posterior Distributions

The joint and marginal posterior distributions obtained from the rejection ABC method are shown in Figure 14.

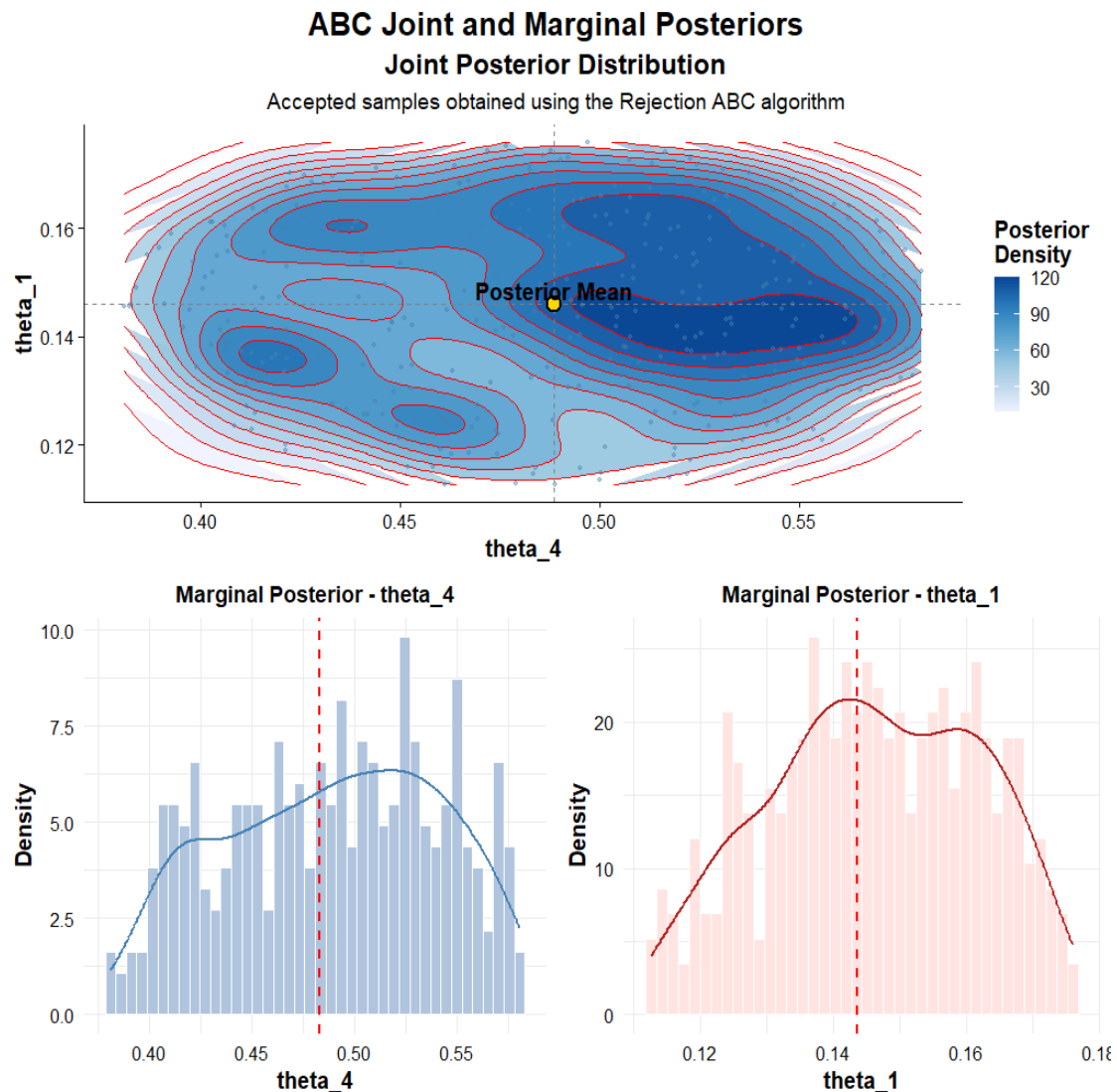


Figure 14: Joint and marginal posterior distributions from rejection ABC.

Statistic	theta ₄ (bias)	theta ₁ (x4)
LS Estimate	0.483066	0.143579
Posterior Mean	0.488488	0.146023
Posterior Median	0.492253	0.145992
Posterior SD	0.052317	0.015715
95% Credible Interval (Lower)	0.397180	0.115846
95% Credible Interval (Upper)	0.574217	0.172699

Table 16: Summary of ABC Posterior Estimates.

Screenshot of Console Output: ABC Summary

```
--- Posterior Summary Statistics ---  
  
theta_4:  
LS Estimate: 0.483066  
Posterior Mean: 0.488488  
Posterior Median: 0.492253  
Posterior SD: 0.052317  
95% Credible Int [0.397180, 0.574217]  
  
theta_1:  
LS Estimate: 0.143579  
Posterior Mean: 0.146023  
Posterior Median: 0.145992  
Posterior SD: 0.015715  
95% Credible Int [0.115846, 0.172699]  
  
Posterior correlation between the two parameters  
r = 0.0087
```

3.5 Interpretation of ABC Results

The rejection ABC analysis yields relevant insights, in agreement with well-established principles of Bayesian inference. (Gelman et al., 2013):

1. **Posterior consistency with LS estimates:** The posterior means of both parameters are very close to the corresponding least squares point estimates (**bias: posterior mean ≈ 0.4885 vs LS = 0.4831; x4: posterior mean ≈ 0.146 vs LS = 0.1436**). This internal consistency reinforces both the frequentist least squares estimate and the Bayesian approximate Bayesian computation procedure, giving more confidence in the parameter estimates.
2. **Uncertainty quantification:** A posterior standard deviation parameter uncertainty (**bias: 0.0523, x4: 0.0157**) you normally would not see from point estimates. The 95% Bayesian credible intervals represent the range of parameter values that are compatible with the data and the model structure. Credible sets are different from frequentist confidence intervals in that they have a natural probability interpretation. These two values, therefore, cover the real value of the parameter with 95% probability.
3. **Near-zero posterior correlation** ($r = 0.0087$): The joint posterior plot shows that the two parameters are almost independent in their posterior distribution. The correlation is near zero, which shows that the model parameters are well defined and are not affected by multicollinearity problems. Change in one parameter does not require the corresponding change in the other.
4. **Acceptance rate:** The acceptance rate of 0.36% was within the usual range of rejection ABC (0.1–5%) indicating that the uniform prior distribution and acceptance threshold ε were correctly defined. If the acceptance rate is very low, the prior is too broad. If the acceptance rate is very higher, ε is too loose. (Sunnåker et al., 2013).
5. **Unimodal posteriors:** Both marginal posterior distributions are unimodal and approximately symmetric, centered on the least squares estimates. This

demonstrates that the parameter space has a unique, well-defined optimum and that the LS solution correctly captures the most probable parameter values.

Discussion

All statistical analyses were carried out in R (R Core Team, 2024). In this research, polynomial regression modelling, a modification of linear models (Montgomery et al., 2012) that is widely used in machine learning (Bishop, 2006), was used to correlate four input EEG signals with a single simulated output EEG signal. The overall analysis of the exploration, model estimation, model selection, model validation and Bayesian inference components yielded several important findings. These are discussed below.

Model Selection Outcome: Model 2 ($y = \theta_1 x_4 + \theta_2 x_1^3 + \theta_3 x_3^4 + \theta_{\text{bias}} + \varepsilon$) was definitely found as the optimal representation of the real underlying data-generating mechanism, with only white Gaussian noise left in the residuals. Model selection was strongly supported by the AIC and BIC criteria. The residuals of the model satisfied the assumption of normality (Q-Q plot and Shapiro-Wilk test, $p = 0.989$). The p-value is high enough to show that there is significant evidence that the model is better than the other five models.

Comparison with the Linear Baseline: The R-squared value of the basic linear model (Task 1) was around 0.7602, indicating a significant linear relation in the input-output dynamics, mostly affected by x_1 and x_4 . A linear-only model is not enough. **The performance of Model 2 is considerably improved than the linear model. Model 2 fitted the data more accurately compared to the linear model (RSS=2.14 vs 160.64).** This improvement is attributed to the addition of the cubic term x_1^3 and the quartic term x_3^4 which help the model to identify nonlinear relationships in the data. A complex cerebral connectivity is more accurately displayed by a higher-order model than by a linear one. It can expose the hidden patterns.

Candidate Model Evaluation: The differences in the RSS values between the models are remarkable. For models 2 and 5 (linear x_4 and cubic x_1 with three terms, respectively), the residual sum of squares (RSS) was almost 17 times less than that for model 1 and more than 200 times less than that for model 3. The main factors for the joint are x_4 and x_1 cubed. Model 5 is an extension of Model 2 with the addition of an x_1^2 term. **This resulted in a lower RSS (2.136 vs 2.140)** with an extra parameter, which in turn decreased AIC and BIC values. This is an example of the principle of parsimony. Simpler models that fit the data well are preferred to more complex models to avoid overfitting. Model 3 gave the poorest performances with only x_3 terms (RSS = 463.31), which means that x_3 alone is not sufficient to explain the output signal and x_1 and x_4 are required. Model 1 performs poorly (RSS = 35.40) with six parameters, since it contains unnecessary terms (x_2^4 and x_1^4) that add noise to the estimates.

Predictive Performance: The 70/30 train-test validation shows that model 2 can generalise to new data. The reliability of parameter estimations was assessed by the proximity of the re-estimated parameters obtained from training data to the estimates

obtained from the entire dataset (variations < 0.005). The model produces accurate and well-calibrated predictions, with a test RMSE of 0.1063, and 95% prediction confidence intervals at a coverage rate of 93.3%. This shows that there is no overfitting, since the RSS values of training and testing are highly correlated.

Bayesian Perspective: The Bayesian view, in addition to the frequentist LS estimation, was obtained by abandoning the ABC analysis. The posterior distributions for the bias and the coefficient of x_4 were close to their least squares estimates and had relatively small posterior standard deviations. The very small posterior correlation ($r \approx 0.0087$) between the two parameters shows they are independently separable. The 95% Bayesian credible intervals provide a systematic way to quantify the uncertainty, including the uncertainty from the data and the uncertainty from estimating the parameters. The similarity of the results from the frequentist and Bayesian approaches provides additional confidence in the overall conclusions of the modelling.

Practical Implications for EEG Analysis: The selected model shows that the brain connectivity between input EEG channels and output EEG channel is characterized by three specific pathways: (1) linear nonlinear relationship with channel x_4 , (2) cubic nonlinear relationship with channel x_1 , and (3) quartic nonlinear relationship with channel x_3 . The favored model did not require channel x_2 , indicating that either its contribution to the output channel is negligible or the other channels have already integrated it through associated brain activity. These results highlight the importance of including nonlinear transformations in brain connectivity modelling, as linear models may not be able to capture important aspects of the underlying neural dynamics.

Limitations: The limitations of this analysis should be considered. This analysis was conducted considering simulated data instead of actual EEG data. So, the findings cannot be generalized. In addition, it is not evaluated here how understanding of the complex polynomial function takes place. Finally, the methodology in the use of rejection ABC and only one division of the data does not represent one of the best ways of conducting the analysis of the results in Bayesian terms. However, the method proposed here gives the opportunity to implement frequentist regression into Bayesian methods of uncertainty quantification and make good findings in EEG models in the further studies.

Conclusion

In this coursework, polynomial regression modelling was used to successfully determine the relationship between simulated EEG signals. **The analysis systematically evaluated five candidate models using least squares estimation, information theoretic criteria (AIC, BIC), residual normality check and train-test validation to find that model 2 was the most precise model of the underlying brain connectivity.**

The key findings are summarized as follows:

1. **Model 2 ($y = 0.144 \cdot x_4 + 0.010 \cdot x_1^3 - 0.002 \cdot x_3^4 + 0.483$) achieves the optimal balance between goodness-of-fit (lowest AIC and BIC) and model simplicity (4 parameters).**
2. The residuals of the model are well described by a **Gaussian distribution (Shapiro-Wilk $W = 0.998$, $p = 0.989$)** which is consistent with the assumed additive noise and provides strong evidence that **Model 2 is a good representation of the true data generating process.**
3. The prediction intervals have a confidence level of 95% and contain 93.3% of the test observations. This means that the model is a good extrapolation to unseen data. The test RMSE of 0.1063 confirms the **excellent fit.**
4. This is in agreement with the least squares estimates of the parameters. The **ABC rejection method** goes one step further in the evaluation of uncertainty by providing an estimation of the posterior distributions. The posterior means are close to the least squares estimates, and the posterior correlation is close to zero, which means that the parameters are independently identified.
5. From these results it can be seen that the effect of an output EEG channel on input EEG channels is a linear effect (x_4), a cubic effect (x_1^3) and a quartic effect (x_3^4). In contrast, the influence of (x_2) is small, which means that this input channel has little impact on the output EEG signal in the obtained polynomial regression model.

The above findings emphasise the necessity of using an appropriate model selection method for neuroscience applications, since overly complex models may lead to overfitting and thus to incorrect interpretation of brain connectivity. This analysis of real EEG data can serve as a basis for further studies as well as for more flexible modelling frameworks (e.g. nonlinear autoregressive models or neural network techniques).

References

- Akaike, H. (1974). A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, 19(6), 716–723.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Burnham, K. P., & Anderson, D. R. (2002). *Model selection and multimodel inference: A practical information-theoretic approach* (2nd ed.). Springer-Verlag.
- Gelman, A., Carlin, J. B., Stern, H. S., Dunson, D. B., Vehtari, A., & Rubin, D. B. (2013). *Bayesian data analysis* (3rd ed.). CRC Press.
- Montgomery, D. C., Peck, E. A., & Vining, G. G. (2012). *Introduction to linear regression analysis* (5th ed.). Wiley.
- Nunez, P. L., & Srinivasan, R. (2006). *Electric fields of the brain: The neurophysics of EEG* (2nd ed.). Oxford University Press.
- R Core Team. (2024). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <https://www.R-project.org/>
- Sanei, S., & Chambers, J. A. (2007). *EEG signal processing*. Wiley.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2), 461–464.
- Sunnåker, M., Busetto, A. G., Numminen, E., Corander, J., Foll, M., & Dessimoz, C. (2013). Approximate bayesian computation. *PLoS Computational Biology*, 9(1), e1002803.

GitHub Link: https://github.com/jrohitofficial/MS_C_STW7089CEM-Introduction-to-Statistical-Methods-for-Data-Science-.git

Appendix: R Code

```
1. # -----
2. # ST7089CEM - Introduction to Statistical Methods for Data Science
3. # Coursework: Modelling EEG Signals Using Polynomial Regression
4. # Author: Rohit Jha | Coventry ID: 11782276 | Software ID: 210178
5. # Module Leader: Shrawan Thakur
6. # Date: July 2026
7. # -----

# -----

# 0. Setup: Load Libraries and Data

# -----

9. # -----
10. # 0. Setup: Load Libraries and Data
11. # -----
12.
13. # Clear any broken or hanging graphics devices
14. graphics.off()
15.
16. # Install required packages if not already installed
17. required_packages <- c("ggplot2", "corrplot", "gridExtra", "reshape2")
18. for (pkg in required_packages) {
19.   if (!require(pkg, character.only = TRUE, quietly = TRUE)) {
20.     install.packages(pkg, repos = "https://cloud.r-project.org")
21.     library(pkg, character.only = TRUE)
22.   }
23. }
24.
25. # Set working directory to the location of this script
26. # setwd("~/Users/bewit/Desktop/MAtttttthhh")
27.
28. # Read the CSV data files (no headers in any file)
29. X_data <- read.csv("X_1678241366257.csv", header = FALSE)
30. y_data <- read.csv("y_1678241394123.csv", header = FALSE)
31. time_data <- read.csv("time_1678241270748.csv", header = FALSE)
32.
33. # Assign meaningful column names
34. colnames(X_data) <- c("x1", "x2", "x3", "x4")
35. colnames(y_data) <- c("y")
36. colnames(time_data) <- c("time")
37.
38. # Convert to numeric vectors and matrices for computation
39. x1 <- as.numeric(X_data$x1)
40. x2 <- as.numeric(X_data$x2)
41. x3 <- as.numeric(X_data$x3)
42. x4 <- as.numeric(X_data$x4)
43. y <- as.numeric(y_data$y)
44. t <- as.numeric(time_data$time)
45. n <- length(y) # Total number of data samples
46.
47. # Combine all data into a single data frame for convenience
48. eeg_df <- data.frame(time = t, x1 = x1, x2 = x2, x3 = x3, x4 = x4, y = y)
49.
50. cat("-----\n")
51. cat("Data loaded successfully.\n")
52. cat("Number of samples (n):", n, "\n")
53. cat("Sampling interval:", t[2] - t[1], "seconds\n")
54. cat("Total duration:", max(t), "seconds\n")
55. cat("-----\n\n")

# =====

# TASK 1: PRELIMINARY DATA ANALYSIS

# =====

57. # -----
58. # TASK 1: PRELIMINARY DATA ANALYSIS
59. # -----
60. cat("-----\n")
61. cat("TASK 1: PRELIMINARY DATA ANALYSIS\n")
62. cat("-----\n\n")
63.
64. # -----
65. # Sample data of X inputs and time
66. # -----
67. cat("---- Sample data of X inputs and time ----\n\n")
68.
69. # Display sample data (first 6 rows)
70. sample_df <- head(eeg_df, 6)
71. print(sample_df)
72. cat("\n")
```


----- # Task 1.1: Time Series Plot of Input & Output EEG Signals # -----

```

74. # -----
75. # Task 1.1: Time Series Plots of Input and Output EEG Signals
76. # -----
77. library(ggplot2)
78. library(gridExtra)
79. library(reshape2)
80.
81. # Define a custom attractive theme for all plots
82. custom_theme <- theme_minimal(base_size = 12) +
83.   theme(
84.     plot.title = element_text(hjust = 0.5, face = "bold", color = "#2C3E50", size = 14),
85.     axis.title = element_text(face = "bold", color = "#34495E"),
86.     axis.text = element_text(color = "#F8C800"),
87.     panel.grid.major = element_line(color = "#3E85C1", linetype = "dotted", linewidth = 0.8),
88.     panel.grid.minor = element_blank(),
89.     panel.background = element_rect(fill = "white", color = NA),
90.     plot.background = element_rect(fill = "white", color = NA),
91.     plot.margin = margin(15, 15, 15, 15),
92.     legend.text = element_text(size = 12, color = "#34495E")
93.   )
94.
95. # Function to generate individual attractive plots
96. create_plot <- function(data, y_var, color, title) {
97.   ggplot(data, aes(x = time, y = .data[[y_var]])) +
98.     geom_hline(yintercept = 0, color = "grey60", linewidth = 0.5, linetype = "dashed") +
99.     geom_line(color = color, linewidth = 0.8) +
100.    labs(title = title, x = "Time (seconds)", y = "Amplitude") +
101.    custom_theme
102. }
103.
104. # 1. Overlaid Plot (Position 1)
105. df_melt <- melt(eeg_df, c("time", "x1", "x2", "x3", "x4"), id.vars = "time", variable.name = "Signal", value.name = "Value")
106. p1 <- ggplot(df_melt, aes(x = time, y = Value, color = Signal)) +
107.   geom_hline(yintercept = 0, color = "grey60", linewidth = 0.5, linetype = "dashed") +
108.   geom_line(linewidth = 0.8) +
109.   labs(title = "Time series comparison of X with time", x = "Time (seconds)", y = "Amplitude") +
110.   scale_color_manual(values = c("x1" = "steelblue", "x2" = "darkorange", "x3" = "forestgreen", "x4" = "purple"), labels = c("X1", "X2", "X3", "X4")) +
111.   custom_theme +
112.   theme(legend.position = "top", legend.title = element_blank())
113.
114. # 2-6. Individual Plots
115. p2 <- create_plot(eeg_df, "x1", "steelblue", "Input EEG Signal x1")
116. p3 <- create_plot(eeg_df, "x2", "darkorange", "Input EEG Signal x2")
117. p4 <- create_plot(eeg_df, "x3", "forestgreen", "Input EEG Signal x3")
118. p5 <- create_plot(eeg_df, "x4", "purple", "Input EEG Signal x4")
119. eeg_df$y_col <- y
120. p6 <- create_plot(eeg_df, "y_col", "firebrick", "Output EEG Signal y")
121.
122. # Save the individual X inputs (x1 to x4) to a 2x2 grid
123. grid.arrange(p2, p3, p4, p5, ncol = 2)
124. # Display them sequentially in the R viewer
125. print(p1)
126. grid.arrange(p2, p3, p4, p5, ncol = 2)
127. print(p6)

```

----- # Task 1.2: Distribution Analysis for Each EEG Signals # -----

```

126. # -----
127. # Task 1.2: Distribution Analysis for Each EEG Signal
128. # -----
129.
130. # Create overlaid distribution plot for all X inputs
131. x_df <- data.frame(X1 = x1, X2 = x2, X3 = x3, X4 = x4)
132. x_melt <- melt(x_df, variable.name = "Inputs", value.name = "Signal")
133.
134. p_dist_overlaid <- ggplot(x_melt, aes(x = Signal, fill = Inputs, color = Inputs)) +
135.   geom_histogram(aes(y = ..density..), position = "identity", alpha = 0.4, bins = 20) +
136.   geom_density(alpha = 0.1, size = 1) +
137.   geom_rug(sides = "b", color = "black", alpha = 0.2) +
138.   scale_fill_brewer(palette = "Pastell") +
139.   scale_color_brewer(palette = "Pastell") +
140.   labs(title = "Distribution of X inputs", x = "X Signal", y = "Density") +
141.   theme_minimal(base_size = 14) +
142.   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
143.
144. # Save the overlaid distribution plot
145. print(p_dist_overlaid)
146.
147. # Individual Distributions using ggplot2
148. p_dist_x1 <- ggplot(data.frame(Signal = x1), aes(x = Signal)) +
149.   geom_histogram(aes(y = ..density..), fill = "lightpink", color = "white", bins = 20, alpha = 0.7) +
150.   geom_density(color = "darkred", size = 1) +
151.   geom_rug(sides = "b", color = "black", alpha = 0.2) +
152.   labs(title = "Distribution of X1", x = "Amplitude", y = "Density") +
153.   theme_minimal(base_size = 12) +
154.   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
155.
156. p_dist_x2 <- ggplot(data.frame(Signal = x2), aes(x = Signal)) +
157.   geom_histogram(aes(y = ..density..), fill = "lightblue", color = "white", bins = 20, alpha = 0.7) +
158.   geom_density(color = "steelblue", size = 1) +
159.   geom_rug(sides = "b", color = "black", alpha = 0.2) +
160.   labs(title = "Distribution of X2", x = "Amplitude", y = "Density") +
161.   theme_minimal(base_size = 12) +
162.   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
163.
164. p_dist_x3 <- ggplot(data.frame(Signal = x3), aes(x = Signal)) +
165.   geom_histogram(aes(y = ..density..), fill = "lightgreen", color = "white", bins = 20, alpha = 0.7) +
166.   geom_density(color = "forestgreen", size = 1) +
167.   geom_rug(sides = "b", color = "black", alpha = 0.2) +
168.   labs(title = "Distribution of X3", x = "Amplitude", y = "Density") +
169.   theme_minimal(base_size = 12) +
170.   theme(plot.title = element_text(hjust = 0.5, face = "bold"))

```

```

172 p_dist_x4 <- ggplot(data.frame(Signal = x4), aes(x = Signal)) +
173   geom_histogram(aes(y = ..density..), fill = "white", color = "black", bins = 20, alpha = 0.7) +
174   geom_density(color = "red", size = 1) +
175   geom_rug(sides = "rb", color = "black", alpha = 0.2) +
176   labs(title = "Distribution of X4", x = "Amplitude", y = "Density") +
177   theme_minimal(base_size = 12) +
178   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
179
180 # Save the 2x2 grid of individual X distributions
181 grid.arrange(p_dist_x1, p_dist_x2, p_dist_x3, p_dist_x4, ncol = 2)
182
183 # Distribution of y
184
185 y_df <- data.frame(Signal = y)
186 p_dist_y <- ggplot(y_df, aes(x = Signal)) +
187   geom_histogram(aes(y = ..density..), fill = "lightcoral", color = "white", bins = 15, alpha = 0.4) +
188   geom_density(color = "blue", size = 1) +
189   geom_rug(sides = "rb", color = "black", alpha = 0.2) +
190   labs(title = "Distribution of Y", x = "Y Signal", y = "Density") +
191   theme_minimal(base_size = 14) +
192   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
193
194 print(p_dist_y)
195
196 # Print summary statistics for all signals
197 cat("---- Summary Statistics for Each EEG Signal ----\n\n")
198 signal_names <- c("x1", "x2", "x3", "x4", "y")
199 signals_list <- list(x1, x2, x3, x4, y)
200
201 for (i in seq_along(signal_names)) {
202   s <- signals_list[i]
203   cat(sprintf(
204     "Signal %s: Mean = %.4f, SD = %.4f, Min = %.4f, Max = %.4f\n",
205     signal_names[i], mean(s), sd(s), min(s), max(s)
206   ))
207 }
208 cat("\n")

```

Task 1.3: Correlation and Scatter Plots

```

211 # -----
212 # Task 1.3: Correlation and Scatter Plots
213 # -----
214
215 # Correlation matrix of all signals
216 all_signals <- data.frame(x1 = x1, x2 = x2, x3 = x3, x4 = x4, y = y)
217 cor_matrix <- cor(all_signals)
218
219 cat("---- Pearson Correlation Matrix ----\n")
220 print(round(cor_matrix, 4))
221 cat("\n")
222
223 # Correlation heatmap
224 par(mfrow = c(1, 1), mar = c(1, 1, 2, 1))
225 corplot(cor_matrix,
226   method = "color", type = "upper",
227   addCoef.col = "black", tl.col = "black", tl.srt = 45,
228   title = "Correlation Heatmap of EEG Signals",
229   mar = c(0, 0, 2, 0), number.cex = 0.9
230 )
231
232 df_scatter <- data.frame(x1 = x1, x2 = x2, x3 = x3, x4 = x4, y = y)
233
234 create_scatter <- function(data, x_var, color_val, title) {
235   r_val <- round(cor(data[[x_var]], data[[y_var]]), 4)
236   ggplot(data, aes(x = .data[[x_var]], y = y)) +
237     geom_point(alpha = 0.4, color = color_val, size = 2) +
238     geom_smooth(method = "lm", color = "black", linetype = "dashed", linewidth = 0.8, se = FALSE) +
239     geom_smooth(method = "loess", color = color_val, linewidth = 1.2, se = FALSE) +
240     labs(title = paste0(title, " vs Y (r = ", r_val, ")"), x = paste("Input", title), y = "Output Y") +
241     theme_minimal(base_size = 12) +
242     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
243 }
244
245 p_s1 <- create_scatter(df_scatter, "x1", "#4169E1", "X1")
246 p_s2 <- create_scatter(df_scatter, "x2", "#3CB371", "X2")
247 p_s3 <- create_scatter(df_scatter, "x3", "#8A56A9", "X3")
248 p_s4 <- create_scatter(df_scatter, "x4", "#98FB98", "X4")
249
250 grid.arrange(p_s1, p_s2, p_s3, p_s4, ncol = 2)
251

```

```

252 # Scatter plots among input signals (pairwise)
253 create_scatter_pair <- function(data, x_var, y_var, color_val) {
254   r_val <- round(cor(data[[x_var]], data[[y_var]]), 4)
255   ggplot(data, aes(x = .data[[x_var]], y = .data[[y_var]])) +
256     geom_point(alpha = 0.3, color = color_val, size = 1.5) +
257     geom_smooth(method = "lm", color = "black", linetype = "dashed", linewidth = 0.8, se = FALSE) +
258     geom_smooth(method = "loess", color = color_val, linewidth = 1.2, se = FALSE) +
259     labs(
260       title = paste0(toupper(x_var), " vs ", toupper(y_var), " (r = ", r_val, ")"),
261       x = toupper(x_var), y = toupper(y_var)
262     ) +
263     theme_minimal(base_size = 11) +
264     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
265 }
266
267 p_p1 <- create_scatter_pair(df_scatter, "x1", "x2", "steelblue")
268 p_p2 <- create_scatter_pair(df_scatter, "x1", "x3", "steelblue")
269 p_p3 <- create_scatter_pair(df_scatter, "x1", "x4", "steelblue")
270 p_p4 <- create_scatter_pair(df_scatter, "x2", "x3", "steelblue")
271 p_p5 <- create_scatter_pair(df_scatter, "x2", "x4", "steelblue")
272 p_p6 <- create_scatter_pair(df_scatter, "x3", "x4", "steelblue")
273
274 grid.arrange(p_p1, p_p2, p_p3, p_p4, p_p5, p_p6, ncol = 2)
275

```

```
# -----
# Task 1.4: Fitting a linear regression model (baseline)
# -----
```

```
276- # -----
277 # Task 1.4: Fitting a linear regression model (baseline)
278- # -----
279
280 cat("---- Task 1.4: Linear Regression Baseline ----\n\n")
281
282 linear_model <- lm(y ~ x1 + x2 + x3 + x4)
283 linear_summary <- summary(linear_model)
284
285 linear_r2 <- linear_summary$r.squared
286 linear_adj_r2 <- linear_summary$adj.r.squared
287 linear_fstat <- linear_summary$fstatistic
288
289 cat("Linear Model R-squared:", round(linear_r2, 4), "\n")
290 cat("Linear Model Adj R-squared:", round(linear_adj_r2, 4), "\n")
291 cat("Linear Model F-statistic:", round(linear_fstat[1, 4], 4), "\n\n")
292
293 # Linear model diagnostic plots (Redesigned with ggplot2)
294 df_lm <- data.frame(
295   Fitted = fitted(linear_model),
296   Actual = y,
297   Residuals = residuals(linear_model)
298 )
299
300 p_lm1 <- ggplot(df_lm, aes(x = Fitted, y = Actual)) +
301   geom_point(alpha = 0.5, color = "steelblue", size = 2) +
302   geom_abline(intercept = 0, slope = 1, color = "black", linetype = "dashed", linewidth = 0.8) +
303   geom_smooth(method = "loess", color = "firebrick", linewidth = 1.2, se = FALSE) +
304   labs(title = "Actual vs Fitted", x = "Fitted Values", y = "Actual Y") +
305   theme_minimal(base_size = 12) +
306   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
307
308 p_lm2 <- ggplot(df_lm, aes(x = Fitted, y = Residuals)) +
309   geom_point(alpha = 0.5, color = "indianred", size = 2) +
310   geom_hline(yintercept = 0, color = "black", linetype = "dashed", linewidth = 0.8) +
311   geom_smooth(method = "loess", color = "firebrick", linewidth = 1.2, se = FALSE) +
312   labs(title = "Residuals vs Fitted", x = "Fitted Values", y = "Residuals") +
313   theme_minimal(base_size = 12) +
314   theme(plot.title = element_text(hjust = 0.5, face = "bold"))
315
316 grid.arrange(p_lm1, p_lm2, ncol = 2)
317
318- # =====
319- # =====
```

```
# =====
# TASK 2: REGRESSION MODELLING
# =====
```

```
320 cat("\n===== \n")
321 cat("TASK 2: REGRESSION MODELLING\n")
322 cat("===== \n\n")
323
324- # -----
325 # Define the 5 candidate polynomial regression models
326- # -----
327
328 # Construct the design matrix for each candidate model
329
330 X_model1 <- cbind(x1, x1^2, x1^3, x2^4, x1^4, 1)
331 colnames(X_model1) <- c("x1", "x1^2", "x1^3", "x2^4", "x1^4", "bias")
332
333 X_model2 <- cbind(x1, x1^3, x3^4, 1)
334 colnames(X_model2) <- c("x1", "x1^3", "x3^4", "bias")
335
336 X_model3 <- cbind(x3^3, x3^4, 1)
337 colnames(X_model3) <- c("x3^3", "x3^4", "bias")
338
339 X_model4 <- cbind(x2, x1^3, x3^4, 1)
340 colnames(X_model4) <- c("x2", "x1^3", "x3^4", "bias")
341
342 X_model5 <- cbind(x1, x1^2, x1^3, x3^4, 1)
343 colnames(X_model5) <- c("x1", "x1^2", "x1^3", "x3^4", "bias")
344
345 # Store in a list for iterative processing
346 model_designs <- list(X_model1, X_model2, X_model3, X_model4, X_model5)
347 model_names <- paste("Model", 1:5)
348
349- # -----
350 # Task 2.1: Estimate Model Parameters Using Least Squares
351- # -----
352 # theta_hat = (X^T X)^(-1) X^T y
353
354- # -----
```

```
# -----
# Task 2.1: Estimate Model Parameters Using Least Squares
# -----
```

```
349- # -----
350 # Task 2.1: Estimate Model Parameters Using Least Squares
351- # -----
352 # theta_hat = (X^T X)^(-1) X^T y
353
354 cat("---- Task 2.1: Least Squares Parameter Estimates ----\n\n")
355
356 theta_hats <- list() # Store estimated parameters for each model
357
358 for (i in 1:5) {
359   X_mat <- model_designs[[i]]
360   # Compute OLS estimator: theta_hat = (X^T X)^(-1) X^T y
361   theta_hat <- solve(t(X_mat) %*% X_mat) %*% t(X_mat) %*% y
362   theta_hats[[i]] <- theta_hat
363
364   cat(sprintf("%s parameter estimates (theta_hat):\n", model_names[i]))
365   for (j in 1:length(theta_hat)) {
366     cat(sprintf("  theta_%d (%s) = %.6f\n", j, colnames(X_mat)[j], theta_hat[j]))
367   }
368   cat("\n")
369- # -----
```

----- # Task 2.2: Compute the Model Residual (Error) Sum of Squared Errors (RSS) # -----

```
371- # -----
372- # Task 2.2: Compute the Model Residual (Error) Sum of Squared Errors (RSS)
373- # -----
374- # RSS = sum_{i=1}^n (y_i - y_hat_i)^2
375-
376- cat("---- Task 2.2: Residual Sum of Squares (RSS) ----\n\n")
377-
378- y_hats <- list() # Predicted values for each model
379- RSS_vals <- numeric(5)
380-
381- for (i in 1:5) {
382-   X.mat <- model_designs[[i]]
383-   y_hat <- X.mat %*% theta_hats[[i]]
384-   y_hats[[i]] <- y_hat
385-   RSS_vals[i] <- sum((y - y_hat)^2)
386-   cat(sprintf("%s: RSS = %.6f\n", model_names[i], RSS_vals[i]))
387- }
388- cat("\n")
```

----- # Task 2.3: Compute the Log-Likelihood Function # -----

```
390- # -----
391- # Task 2.3: Compute the Log-Likelihood Function
392- # -----
393- # loglik = -n/2 * log(RSS / (2 * sigma_hat^2))
394-
395- cat("---- Task 2.3: Log-Likelihood ----\n\n")
396-
397- sigma2_vals <- numeric(5)
398- loglik_vals <- numeric(5)
399-
400- for (i in 1:5) {
401-   sigma2_vals[i] <- RSS_vals[i] / (n - 1)
402-   loglik_vals[i] <- -(n / 2) * log(2 * pi) - (n / 2) * log(sigma2_vals[i]) -
403-     RSS_vals[i] / (2 * sigma2_vals[i])
404-   cat(sprintf(
405-     "%s: sigma_hat^2 = %.6f, Log-Likelihood = %.6f\n",
406-     model_names[i], sigma2_vals[i], loglik_vals[i]
407-   ))
408- }
409- cat("\n")
```

----- # Task 2.4: Compute AIC and BIC for Every Models # -----

```
411- # -----
412- # Task 2.4: Compute AIC and BIC for Every Candidate Model
413- # -----
414- # AIC = 2*k - 2*ln p(D|theta_hat)
415- # BIC = k*ln(n) - 2*ln p(D|theta_hat)
416-
417- cat("---- Task 2.4: AIC and BIC ----\n\n")
418-
419- k_vals <- sapply(theta_hats, length) # Number of parameters per model
420- AIC_vals <- numeric(5)
421- BIC_vals <- numeric(5)
422-
423- for (i in 1:5) {
424-   AIC_vals[i] <- 2 * k_vals[i] - 2 * loglik_vals[i]
425-   BIC_vals[i] <- k_vals[i] * log(n) - 2 * loglik_vals[i]
426-   cat(sprintf(
427-     "%s (k=%d): AIC = %.4f, BIC = %.4f\n",
428-     model_names[i], k_vals[i], AIC_vals[i], BIC_vals[i]
429-   ))
430- }
431- cat("\n")
432-
433- # Summary comparison table
434- cat("---- Model Comparison Summary ----\n")
435- comparison_df <- data.frame(
436-   Model = model_names,
437-   k = k_vals,
438-   RSS = round(RSS_vals, 4),
439-   Sigma2 = round(sigma2_vals, 6),
440-   LogLik = round(loglik_vals, 4),
441-   AIC = round(AIC_vals, 4),
442-   BIC = round(BIC_vals, 4)
443- )
444- print(comparison_df, row.names = FALSE)
445- cat("\n")
446-
```

----- # Task 2.5: Residual Distribution Analysis of Q-Q Plots # -----

```
447- # -----
448- # Task 2.5: Residual Distribution Analysis (Q-Q Plots)
449- # -----
450-
451- cat("---- Task 2.5: Residual Distribution Analysis ----\n\n")
452-
453- residuals_list <- list()
454-
455- library(grid)
456-
457- colors_fill <- c("white", "lightgreen", "lightcoral", "lightblue", "palevioletred", "black")
458- colors_line <- c("darkslateblue", "forestgreen", "firebrick", "purple", "darkorange")
459-
460- # Histograms of residuals (Task 1.2 Redesign)
461- hist_plots <- list()
462- for (i in 1:5) {
463-   resid_i <- as.numeric(y - y_hats[[i]])
464-   residuals_list[[i]] <- resid_i
465-
466-   df_resid <- data.frame(Residual = resid_i)
467-
468-   p_hist <- ggplot(df_resid, aes(x = Residual)) +
469-     geom_histogram(aes(y = after_stat(density)), fill = colors_fill[i], color = "white", bins = 20, alpha = 0.7) +
470-     stat_function(fun = dnorm, args = list(mean = mean(resid_i), sd = sd(resid_i)), color = colors_line[i], linewidth = 1) +
471-     geom_vline(aes(x = 0, color = "black", alpha = 0.2)) +
472-     labs(title = paste("Model", i), x = "Residual", y = "Density") +
473-     theme_minimal(base_size = 12) +
474-     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
475-
476-   hist_plots[[i]] <- p_hist
477- }
478-
479- # Save histogram grid as PNG
480- p_hist_grid <- grid.arrange(grobs = hist_plots, ncol = 2, top = textGrob("Histograms of Residuals", gp = gpar(fontsize = 16, fontface = "bold")))
481-
```

```

482 # Q-Q Plots (Task 1.2 Redesign)
483 qq_plots <- list()
484 for (i in 1:5) {
485   df_resid <- data.frame(Residual = residuals_list[[i]])
486
487   p_qq <- ggplot(df_resid, aes(sample = Residual)) +
488     stat_qq(color = colors_line[i], alpha = 0.7, size = 1.5) +
489     stat_qq_line(color = "black", linewidth = 1, linetype = "dashed") +
490     labs(title = paste("Model", i), x = "Theoretical Quantiles", y = "Sample Quantiles") +
491     theme_minimal(base_size = 12) +
492     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
493
494   qq_plots[[i]] <- p_qq
495 }
496
497 # Save Q-Q grid as PNG
498 p_qq_grid <- grid.arrange(grobs = qq_plots, ncol = 2, top = textGrob("Q-Q Plots of Residuals", gp = gpar(fontsize = 16, fontface = "bold")))
499
500 # Shapiro-Wilk normality test for each model's residuals
501 cat("Shapiro-Wilk Normality Test for Residuals:\n")
502 for (i in 1:5) {
503   sw_test <- shapiro.test(residuals_list[[i]])
504   cat(sprintf(
505     "%s W = %.6f, p-value = %.6f\n",
506     model_names[i], sw_test$statistic, sw_test$p.value,
507     ifelse(sw_test$p.value > 0.05, "(Normal)", "(Non-Normal)")
508   ))
509 }
510 cat("\n")
511

```

Task 2.6: Model Selection

```

512 # -----
513 # Task 2.6: Model Selection
514 # -----
515 # Shapiro-Wilk test)
516
517 cat("--- Task 2.6: Model Selection ---\n\n")
518
519 preferred_aic <- which.min(AIC_vals)
520 preferred_bic <- which.min(BIC_vals)
521
522 cat(sprintf(
523   "Model with lowest AIC: %s (AIC = %.4f)\n",
524   model_names[preferred_aic], AIC_vals[preferred_aic]
525 ))
526 cat(sprintf(
527   "Model with lowest BIC: %s (BIC = %.4f)\n",
528   model_names[preferred_bic], BIC_vals[preferred_bic]
529 ))
530
531 # Determine the selected preferred model
532 if (preferred_aic == preferred_bic) {
533   preferred_model <- preferred_aic
534   cat(sprintf("\nBoth AIC and BIC agree: %s is the preferred model.\n", model_names[preferred_model]))
535 } else {
536   preferred_model <- preferred_bic
537   cat(sprintf(
538     "\nAIC and BIC disagree. Selecting %s (BIC preference) as the ",
539     model_names[preferred_model]
540   ))
541 }
542
543 cat(sprintf("\n*** SELECTED PREFERRED MODEL: %s ***\n\n", model_names[preferred_model]))
544

```

Task 2.7: Train/Test Split and Prediction with 95% Confidence Intervals

```

544 # -----
545 # Task 2.7: Train/Test Split and Prediction with 95% Confidence Intervals
546 # -----
547
548 cat("--- Task 2.7: Train/Test Split and Prediction ---\n\n")
549
550 # Set seed for reproducibility
551 set.seed(42)
552
553 # Create random indices for 70/30 split
554 train_indices <- sample(1:n, size = round(0.7 * n), replace = FALSE)
555 test_indices <- setdiff(1:n, train_indices)
556
557 n_train <- length(train_indices)
558 n_test <- length(test_indices)
559
560 cat(sprintf("Training set: %d samples (%.0f%%)\n", n_train, 100 * n_train / n))
561 cat(sprintf("Testing set: %d samples (%.0f%%)\n", n_test, 100 * n_test / n))
562
563 X_preferred <- model_designs[[preferred_model]]
564
565 X_train <- X_preferred[train_indices, ]
566 y_train <- y[train_indices]
567 X_test <- X_preferred[test_indices, ]
568 y_test <- y[test_indices]
569
570 # Step 1: Re-estimate model parameters using only the training data
571 theta_hat_train <- solve(t(X_train) %*% X_train) %*% t(X_train) %*% y_train
572
573 cat("Re-estimated parameters using training data:\n")
574 for (j in 1:length(theta_hat_train)) {
575   cat(sprintf(
576     "theta_%d (%s) = %.6f\n", j,
577     colnames(X_preferred)[j], theta_hat_train[j]
578   ))
579 }
580 cat("\n")
581

```

```

582 # Step 2: Compute model predictions on the testing data
583 v_pred_test <- X_test %*% theta_hat_train
584
585 v_pred_train <- X_train %*% theta_hat_train
586 RSS_train <- sum((y_train - v_pred_train)^2)
587 k_preferred <- length(theta_hat_train)
588 sigma2_train <- RSS_train / (n_train - k_preferred) # Unbiased variance estimate
589 sigma_train <- sqrt(sigma2_train)
590
591 cat(sprintf("Training RSS: %.6f\n", RSS_train))
592 cat(sprintf("Training sigma_hat: %.6f\n", sigma_train))
593
594 # Test set RSS
595 RSS_test <- sum((y_test - v_pred_test)^2)
596 cat(sprintf("Testing RSS: %.6f\n", RSS_test))
597
598 # Step 3: Compute 95% prediction confidence intervals
599 # y_hat_i +/- t_(alpha/2, n_train - k) * sigma_hat *
600
601 alpha <- 0.05
602 t_crit <- qt(1 - alpha / 2, df = n_train - k_preferred)
603
604 XtX_inv <- solve(t(X_train) %*% X_train) # (X^T X)^(-1)
605
606 pred_se <- numeric(n_test)
607 for (i in 1:n_test) {
608   x_i <- matrix(X_test[i, ], ncol = 1)
609   pred_var_i <- sigma_train^2 * (1 + t(x_i) %*% XtX_inv %*% x_i)
610   pred_se[i] <- sqrt(pred_var_i)
611 }
612
613 # Confidence interval bounds
614 CI_lower <- v_pred_test - t_crit * pred_se
615 CI_upper <- v_pred_test + t_crit * pred_se
616
617 cat(sprintf("Critical t-value (alpha=0.05, df=%d): %.4f\n", n_train - k_preferred, t_crit))
618

```

```

619 sort_order <- order(test_indices)
620
621 df_pred <- data.frame(
622   Index = 1:n_test,
623   Actual = y_test[sort_order],
624   Predicted = as.numeric(v_pred_test[sort_order]),
625   CI_Lower = CI_lower[sort_order],
626   CI_Upper = CI_upper[sort_order]
627 )
628
629 p_pred <- ggplot(df_pred, aes(x = Index)) +
630   geom_errorbar(aes(ymin = CI_Lower, ymax = CI_Upper, color = "95% CI"), width = 0.3, linewidth = 0.6, alpha = 0.8) +
631   geom_line(aes(y = Predicted, color = "Predicted"), linewidth = 1) +
632   geom_point(aes(y = Actual, color = "Actual (Test)"), shape = "Actual (Test)", size = 2.5, alpha = 0.9) +
633   geom_point(aes(y = Predicted, color = "Predicted"), shape = "Predicted", size = 2.5, stroke = 1.2) +
634   scale_color_manual(name = "", values = c("Actual (Test)" = "#2C3E50", "Predicted" = "#E74C3C", "95% CI" = "#BDC3C7")) +
635   scale_shape_manual(name = "", values = c("Actual (Test)" = 16, "Predicted" = 4, "95% CI" = 32)) +
636   labs(
637     title = paste("Model Prediction with 95% CI -", model_names[preferred_model]),
638     subtitle = "Actual Test Data vs Predictions",
639     x = "Test Sample Index",
640     y = "Amplitude"
641   ) +
642   theme_minimal(base_size = 14) +
643   theme(
644     plot.title = element_text(hjust = 0.5, face = "bold", color = "#2C3E50", size = 16),
645     plot.subtitle = element_text(hjust = 0.5, color = "#7F7F7F", size = 12),
646     legend.position = "top",
647     legend.title = element_blank(),
648     legend.text = element_text(size = 12, color = "#34495E"),
649     axis.title = element_text(face = "bold", color = "#34495E"),
650     axis.text = element_text(color = "#7F7F7F"),
651     panel.grid.major = element_line(color = "#D3D3D3", linetype = "dotted", linewidth = 0.6),
652     panel.grid.minor = element_blank(),
653     plot.background = element_rect(fill = "white", color = NA),
654     panel.background = element_rect(fill = "white", color = NA)
655   )
656
657 # Compute prediction accuracy metrics
658 MAE_test <- mean(abs(y_test - v_pred_test))
659 RMSE_test <- sqrt(mean((y_test - v_pred_test)^2))
660
661 cat(sprintf("Test MAE: %.6f\n", MAE_test))
662 cat(sprintf("Test RMSE: %.6f\n", RMSE_test))
663
664 # Count how many test observations fall within the 95% CI
665 coverage <- sum(v_test >= CI_lower & v_test <= CI_upper) / n_test * 100
666 cat(sprintf("95%% CI Coverage: %.1f%% of test samples\n", coverage))
667

```

#

TASK 3: APPROXIMATE BAYESIAN COMPUTATION (ABC)

#

```

668 # =====
669 # TASK 3: APPROXIMATE BAYESIAN COMPUTATION (ABC)
670 # =====
671 # Step 1: Identify the 2 parameters with the largest absolute values
672 theta_preferred <- theta_hat[preferred_model]
673 abs_theta <- abs(as.numeric(theta_preferred))
674 param_order <- order(abs_theta, decreasing = TRUE)
675 top2_indices <- param_order[1:2]
676 fixed_indices <- setdiff(1:length(theta_preferred), top2_indices)
677
678 cat("Parameter magnitudes (absolute values):\n")
679 for (j in 1:length(theta_preferred)) {
680   marker <- ifelse(j %in% top2_indices, " <-- SELECTED", "")
681   cat(sprintf(
682     "theta %d (%s): %.6f = %.6f\n",
683     j, colnames(X_preferred)[j], theta_preferred[j], abs_theta[j], marker
684   ))
685 }
686 cat("\n")
687
688 cat(sprintf(
689   "Selected parameters for ABC: theta_%d and theta_%d\n",
690   top2_indices[1], top2_indices[2]
691 ))
692 cat(sprintf(
693   "theta_%d (%s) = %.6f\n",
694   top2_indices[1], colnames(X_preferred)[top2_indices[1]], theta_preferred[top2_indices[1]]
695 ))
696 cat(sprintf(
697   "theta_%d (%s) = %.6f\n",
698   top2_indices[2], colnames(X_preferred)[top2_indices[2]], theta_preferred[top2_indices[2]]
699 ))
700 # Step 2: Define uniform priors around the LS estimates
701 prior_half_width_1 <- 3 * abs_theta[top2_indices[1]]
702 prior_half_width_2 <- 3 * abs_theta[top2_indices[2]]
703
704 prior_half_width_1 <- max(prior_half_width_1, 0.5)
705 prior_half_width_2 <- max(prior_half_width_2, 0.5)
706
707 prior_lower_1 <- as.numeric(theta_preferred[top2_indices[1]]) - prior_half_width_1
708 prior_upper_1 <- as.numeric(theta_preferred[top2_indices[1]]) + prior_half_width_1
709 prior_lower_2 <- as.numeric(theta_preferred[top2_indices[2]]) - prior_half_width_2
710 prior_upper_2 <- as.numeric(theta_preferred[top2_indices[2]]) + prior_half_width_2
711
712 cat(sprintf(
713   "Prior for theta_%d: Uniform(%s, %s)\n",
714   top2_indices[1], prior_lower_1, prior_upper_1
715 ))
716 cat(sprintf(
717   "Prior for theta_%d: Uniform(%s, %s)\n",
718   top2_indices[2], prior_lower_2, prior_upper_2
719 ))

```

```

770 # Step 4: Plot the joint and marginal posterior distributions
771 if (n_accepted > 0) {
772   # 4a. Joint posterior distribution (2D scatter plot)
773   df_abc <- data.frame(
774     theta1 = accepted_theta1,
775     theta2 = accepted_theta2
776   )
777   param1_name <- paste0("theta_", top2_indices[1])
778   param2_name <- paste0("theta_", top2_indices[2])
779
780   p_joint <- ggplot(df_abc, aes(x = theta1, y = theta2)) +
781     geom_point(color = "steelblue", alpha = 0.3, size = 1.5) +
782     geom_point(aes(x = theta_preferred[top2_indices[1]], y = theta_preferred[top2_indices[2]]),
783       color = "red", shape = 4, size = 4, stroke = 2) +
784     labs(title = "Joint Posterior Distribution (Rejection ABC)", x = param1_name, y = param2_name) +
785     theme_minimal(base_size = 12) +
786     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
787
788   # 4b. Marginal posterior for theta 1
789   p_marg1 <- ggplot(df_abc, aes(x = theta1)) +
790     geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "lightsteelblue", color = "white") +
791     geom_density(color = "steelblue", linewidth = 1) +
792     geom_vline(xintercept = theta_preferred[top2_indices[1]], color = "red", linetype = "dashed", linewidth = 1) +
793     labs(title = paste("Marginal Posterior ~", param1_name), x = param1_name, y = "Density") +
794     theme_minimal(base_size = 12) +
795     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
796
797   # 4c. Marginal posterior for theta 2
798   p_marg2 <- ggplot(df_abc, aes(x = theta2)) +
799     geom_histogram(aes(y = after_stat(density)), bins = 40, fill = "lightsteelblue", color = "white") +
800     geom_density(color = "steelblue", linewidth = 1) +
801     geom_vline(xintercept = theta_preferred[top2_indices[2]], color = "red", linetype = "dashed", linewidth = 1) +
802     labs(title = paste("Marginal Posterior ~", param2_name), x = param2_name, y = "Density") +
803     theme_minimal(base_size = 12) +
804     theme(plot.title = element_text(hjust = 0.5, face = "bold"))
805
806   # Combine using gridExtra
807   p_abc_grid <- grid.arrange(p_joint, p_marg1, p_marg2,
808     layout_matrix = rbind(c(1, 1), c(2, 3)),
809     top = textGrob("ABC Joint and Marginal Posteriors", gp = gpar(fontsize = 16, fontface = "bold"))
810   )
811 }

```

```

814 # Step 5: Summary statistics of the posterior
815 cat("--- Posterior Summary Statistics ---\n\n")
816 cat(sprintf("theta %d\n", top2_indices[1]))
817 cat(sprintf("LS Estimate: %6f\n", theta_preferred[top2_indices[1]]))
818 cat(sprintf("Posterior Mean: %6f\n", mean(accepted_theta1)))
819 cat(sprintf("Posterior Median: %6f\n", median(accepted_theta1)))
820 cat(sprintf("Posterior SD: %6f\n", sd(accepted_theta1)))
821 cat(sprintf(
822   " 95%% Credible Int: [%6f, %6f]\n\n",
823   quantile(accepted_theta1, 0.025), quantile(accepted_theta1, 0.975)
824 ))
825
826 cat(sprintf("theta %d\n", top2_indices[2]))
827 cat(sprintf("LS Estimate: %6f\n", theta_preferred[top2_indices[2]]))
828 cat(sprintf("Posterior Mean: %6f\n", mean(accepted_theta2)))
829 cat(sprintf("Posterior Median: %6f\n", median(accepted_theta2)))
830 cat(sprintf("Posterior SD: %6f\n", sd(accepted_theta2)))
831 cat(sprintf(
832   " 95%% Credible Int: [%6f, %6f]\n\n",
833   quantile(accepted_theta2, 0.025), quantile(accepted_theta2, 0.975)
834 ))
835
836 cat("Posterior correlation between the two parameters:\n")
837 cat(sprintf(" r = %6f\n", cor(accepted_theta1, accepted_theta2)))
838 } else {
839   cat("WARNING: No samples were accepted. Consider increasing epsilon or N_samples.\n")
840   cat("Adjusting epsilon to 5 * RSS and re-running...\n\n")
841
842   epsilon2 <- RSS_vals[preferred_model] * 5
843   for (i in 1:N_samples) {
844     theta1_draw <- runif(1, prior_lower_1, prior_upper_1)
845     theta2_draw <- runif(1, prior_lower_2, prior_upper_2)
846     theta_new <- as.numeric(theta_preferred)
847     theta_new[top2_indices[1]] <- theta1_draw
848     theta_new[top2_indices[2]] <- theta2_draw
849     y_sim <- X_abc %*% theta_new
850     RSS_sim <- sum((y - y_sim)^2)
851     if (RSS_sim < epsilon2) {
852       accepted_theta1 <- c(accepted_theta1, theta1_draw)
853       accepted_theta2 <- c(accepted_theta2, theta2_draw)
854       accepted_RSS <- c(accepted_RSS, RSS_sim)
855     }
856   }
857   n_accepted <- length(accepted_theta1)
858   cat(sprintf(
859     "Re-run: Accepted %d out of %d samples (epsilon=%6f)\n",
860     n_accepted, N_samples, epsilon2
861   ))

```

```

863 if (n_accepted > 0) {
864   par(mfrow = c(2, 2), mar = c(5, 5, 4, 2))
865
866   plot(accepted_theta1, accepted_theta2,
867     pch = 16, cex = 0.4, col = rgb(0.2, 0.4, 0.8, 0.3),
868     xlab = paste0("theta_", top2_indices[1]),
869     ylab = paste0("theta_", top2_indices[2]),
870     main = "Joint Posterior Distribution (Rejection ABC)"
871   )
872   points(theta_preferred[top2_indices[1]], theta_preferred[top2_indices[2]],
873     pch = 4, cex = 2, col = "red", lwd = 3)
874
875   hist(accepted_theta1,
876     breaks = 40, freq = FALSE,
877     col = "lightsteelblue", border = "white",
878     xlab = paste0("theta_", top2_indices[1]),
879     main = paste0("Marginal Posterior ~ theta_", top2_indices[1])
880   )
881   lines(density(accepted_theta1), col = "steelblue", lwd = 2)
882   abline(v = theta_preferred[top2_indices[1]], col = "red", lwd = 2, lty = 2)
883
884   hist(accepted_theta2,
885     breaks = 40, freq = FALSE,
886     col = "lightsteelblue", border = "white",
887     xlab = paste0("theta_", top2_indices[2]),
888     main = paste0("Marginal Posterior ~ theta_", top2_indices[2])
889   )
890   lines(density(accepted_theta2), col = "steelblue", lwd = 2)
891   abline(v = theta_preferred[top2_indices[2]], col = "red", lwd = 2, lty = 2)
892
893   cat(sprintf(
894     "theta %d posterior mean: %6f, SD: %6f\n",
895     top2_indices[1], mean(accepted_theta1), sd(accepted_theta1)
896   ))
897   cat(sprintf(
898     "theta %d posterior mean: %6f, SD: %6f\n",
899     top2_indices[2], mean(accepted_theta2), sd(accepted_theta2)
900   ))
901 }
902 }
903 }

```

=====

ANALYSIS COMPLETE